

V3.9 — Data Collection Plan (single, organized, binding)

How this document is organized (rebuilt 2026-08-12). This file merges the two specs that previously lived separately:

1. **PART 1 — THE FINAL PLAN (binding).** The complete, current, executable contract: what we collect, how we pay, how we keep it honest, what changes in code, the GO gates, and the step-by-step runbook. This is the **ONLY** part you need to execute. (Previously V3.9_FINAL_PLAN.md, now archived under `archive/`.)
2. **PART 2 — DEEP-DIVE ADDENDA (detail, preserved).** The full worked detail behind each part of the final plan — sampling economics, probability honesty, weather data layer, evaluation suite, anchor protocol, window experiments, calendar, diagnostics, config knobs, model ladder + graph taxonomy. Everything that makes the plan defensible, preserved nearly verbatim. Each addendum is cross-linked to the PART 1 section it supports.
3. **PART 3 — REVISION HISTORY (why it is this way).** How the plan evolved V3.2 → V3.9-f.1: one entry per review, with the claim-by-claim adjudications and the “what we deliberately did not change” frozen lists. This answers the question “**which V3.x do we use?**” — we use **ONE** document, this one; V3.2–V3.8 are revision history, not separate plans.

Removed as redundant during organization: §15 (principle-aligned checklist — a restatement of §§6/8/9/10/13/19, which live in PART 2) and §33 (first-month calendar summary — a duplicate of the full §25 calendar, moved to PART 2 as DD-N). The §27 gates list is superseded by the final GO gates (PART 1 §16) and §24 contents merged into the window-experiment addendum (DD-M). Nothing else was deleted — all remaining detail is preserved in PART 2 or PART 3.

PART 1 — THE FINAL PLAN (binding)

The single, complete, executable contract. Read this part to run the project. Its internal numbering (§1–§22) is the canonical one. Supersedes every prior revision.

Status: V3.9-f.8 PRE-FREEZE PATCH — architecture LOCKED, manifest values MEASURE→FREEZE pending. Collection WAIT until Gate 0 + gates 1–5 + FREEZE pass. DO NOT start Phase 6. Built from reviews 1–12 (ChatGPT1–4, CGTAnalysis5–9, -9

follow-up, 11, and the external SJSU/SDSU-grounded review adjudicated in V3_CollectionStrategy2.md §47) plus the A19/A30 deep audits (ChatGPTv3_A19_1, chatGPTv3_A30_1 #70 77-item checklist, chatGPTv3_A30_2 FINAL MASTER, cgtAnalysis13) plus A30_3 10 consistency fixes. This is the **V3.9-f.8 execution plan — deliberately NOT V3.10**. Once the gates + FREEZE pass we stop revising and let the run produce the evidence. See §22 V3.9-f.8 adjudication for change table.

This file is the ONLY normative specification. V3_CollectionStrategy2.md is the full revision history; where anything there conflicts with this file, this file governs (see §18 for the contradiction map).

1. How to read this plan

- **R#** = code delta, **S#** = schema/pipeline delta, **G#** = GO gate. All must complete before the 60k starts.
- Everything below is the **final, integrated decision**. No section here is a “menu” — V3.1→V3.9 were cumulative revisions of the same plan; this file is the frozen end-state.
- **This file is the ONLY document you need to read to execute.** All other files in AugMDnotes/ (V3_CollectionStrategy*.md, CGTAnalysis*.md, ChatGptAnalysis*.md, V3_WebhookExtractionPlan.md) are history and adjudication records — cited only for provenance, never required for the run.
- Dates/times are UTC unless noted.
- The plan answers, in order: **what we collect (§2–§9)**, **how we pay (§10–§11)**, **how we keep it honest (§12–§14)**, **what changes in code (§15)**, **what must pass before we start (§16)**, **how we run it step-by-step (§17)**, and **why we trust it (§19)**.

2. Locked architecture (end-to-end)

AeroDataBox (maxDeliveryRetries = 0) ← webhook POST
 1 credit per flight item, deducted on SEND

raw immutable notification envelope (S4 - raw payload + SHA-256, never overwritten)

flight_events (append-only per delivery + per observation timestamp) current flight_status
 every field preserved incl. live location (lat/lon/alt/gs/track/vsi/reportedAt)
 NEVER reduced to "latest location" before storing

provider-observable prediction population (S1, FIDS/schedule)

Billing-cycle alignment (A31 §38): The 31-day run MUST start at the beginning of a billing cycle (or document the offset). If the run spans two billing cycles, the daily cap (1,900) applies per cycle, not per calendar day. The 60,000 monthly entitlement resets at the billing cycle boundary — do NOT assume 60,000 is available for the entire run if it spans two cycles. Verify billing cycle dates at Gate 0 and record `billing_cycle_start_date`, `billing_cycle_end_date`, `monthly_entitlement` in the manifest. If the run cannot align with a single billing cycle, split the budget across cycles and track separately.

Run start date freeze (A31 §79): The run start date MUST be frozen at FREEZE (before Phase 6 begins). Record `phase6_start_date` in manifest. Once frozen, the start date CANNOT be changed without re-running the entire FREEZE process. The start date determines: (a) billing cycle alignment, (b) weather archive coverage, (c) historical store bootstrap period, (d) evaluation cutoff dates. If the start date is delayed (e.g., Gate 0.5 fails), the FREEZE must be re-run with the new start date. - **Flight-Alert credit balance** (credits) — dedicated balance, does not expire, pauses at 0; spent only by webhook deliveries: **1 credit per flight item per delivery attempt, deducted on SEND (not delivery), each retry costs another credit.** Refilled via POST /subscriptions/balance/refill at **1 credit = 1 API unit**. Per-refill and balance-cap limits depend on the pricing plan — verify at Gate 0.

3.2 The explicit partition of the 60,000 monthly API units (Gate 0)

60,000 API units (monthly entitlement - VERIFY the actual plan at Gate 0)

Alert-credit refill (1 unit → 1 credit) = 58,900 units → 58,900 credits

Pre-run allocation (A31 §37):

Gate 0 verification	= ~10 credits (1cr=1u refill test)
Gate 0.5 pilot	= ~50-100 credits (1 airport × 1 window)
Gate 1 coverage	= 0 (free endpoint)
Gate 2 anchor probes	= ~100-200 credits (2 probes × 2h)
Gate 3 canary	= ~10-20 credits (1 batch)
Gate 4 cap/reliability	= ~150-320 credits (cap test)
Gate 5 FIDS funnel	= ~200-400 credits (4 airports × 3 horizons × 2u)
FREEZE manifest	= 0 (no credits)

TOTAL pre-run = ~520-1,050 credits (contingency budget)

Spendable experimental envelope = 57,900 credits (58,900 - 1,000 floor)

(1,900 × 31 nominal, minus the permanently reserved 1,000;
the ONLY spendable experimental quota - the true binding
total-invariant is this 57,900, not 58,900)

Permanent balance floor = ADB_RESERVE_CREDITS = 1,000 credits

(carved out of the 58,900 refill; controller refuses to
intentionally spend the balance below 1,000; doubles as the

emergency application reserve)

Census + REST budget 1,000 API units (FIDS 2/call for S1 + anchor pro
census spend is tracked against this line, NEVER against the refill line

Unallocated mathematical remainder = 100 API units (never used experimentally)

Arithmetic check (no double-counting): the 1,000 floor is INSIDE the 58,900 credit refill, not a third add-on. So: 57,900 spendable + 1,000 floor (credits) + 1,000 REST/census + 100 unallocated = 60,000 . **Maximum actual experimental spend = 57,900 credits**, never 58,900. Any report that says “58,900 spend” means *refill size*, which includes the reserved 1,000; the spendable number is always 57,900.

Rules: - The spendable experimental envelope is in **credits** (57,900); the FIDS/census spend is in **API units** (1,000) and comes out of the REST line. They are NOT the same pool — this resolves the contradiction the review found.

- The daily 1,900 cap is the per-day **ceiling**; the 57,900 envelope is the run-total **binding invariant**. On the final day(s), the scheduler sizes the window down automatically so total realized spend never exceeds 57,900 (e.g. days 1–30 $\times$ 1,900 = 57,000 $\rightarrow$ day 31 is capped at the remaining 900).

- Track BOTH the RapidAPI usage page (units) and the Flight-Alert balance (credits). No manual REST-heavy UI actions (Rescore / Simulate / “Rescore all”) during the run.

- **Gate 0 (new):** verify (a) actual plan + monthly units, (b) refill conversion 1 unit = 1 credit, (c) per-refill + balance caps, (d) census spend is budgeted on the REST line — all before any FIDS call.

3.3 Three cap concepts (never conflated)

1. **Estimated reservation (before):** `daily_budget_remaining = 1900 - credits_actually_consumed_today` (from the `adb_ingest_events` ledger). Estimate of what a start is allowed to spend, not a record of spend.
2. **Actual spend (during):** watchdog stops on observed spend. `SOFT_STOP = 1900 - ADB_DAILY_SOFT_STOP_MARGIN` (default 50, tuned from the canary's worst un-settled burst). `HARD_CAP = 1900`; any overshoot flags the batch `reconciliation_status='MISMATCH'`.
3. **Post-batch reconciliation (after):** `C_external = balance_before - balance_after` vs `C_internal =  $\Sigma$  notification_items`; `| $\Delta$ | tolerance` $\rightarrow$ `PASS`, else `MISMATCH`. Only meaningful on an exclusive subscription set (R1).

3.4 Enforced by code (already implemented: V3.9 / §34-Q)

- `adb_collection_batches`: `balance_before/after`, `credits_consumed_actual`, `credits_consumed_internal`, `notification_items_received`, `rows stored/inserted/updated`, `delivery failures`, `reconciliation status`.

- **adb_ingest_events**: one immutable row per webhook delivery (subscription_id, batch_id, received_at, items, stored/inserted/updated/ skipped, credits_remaining, delivery_failure, error).
- Daily-cap math uses **notification items from the ledger**, not row counts.

4. Sampling frame (measured, stratified)

Element	Decision
Universe	AeroDataBox-covered airports per feed, measured via GET <code>/api/v1/collection/coverage</code> (free) → universeCount , catalogInUniverse , universeNotInCatalog
Frame	universe feed-eligible; keep every eligible airport, including zero-yield ones (tracked, never dropped); only coverage-failed airports leave the frame
Zero-yield triage (restored)	zero_yield_once → keep + re-observe; zero_yield_repeated (2 empties) → keep, down-weight candidate; zero_yield_persistent → keep but excluded from adaptive down-weighting's <i>evidence</i> , eligible for manual review. One empty observation is never treated as evidence the airport is useless (could be low traffic, wrong time, temporary outage, schedule effect, or provider issue). Only coverage-failed airports (probe/feed errors, not empty observations) leave the frame
Primary strata	traffic tier × macro-region only (crossing more would explode cell count)
Balancing variables (reported within strata, NOT crossed)	network degree, <i>intl/domestic</i> , <i>carrier diversity</i> , time zone — from a FIXED reference snapshot at frame-build time, never from the recursive current sample

Element	Decision
Catalog	catalog build script; regenerates when coverage refreshes (12 h cache)
Tier mix	{HUB:1, MID:2, REGIONAL:1} per batch — rotating anchor consumes the HUB slot when anchor enabled (see §8.4)
Unit of prediction	a flight-leg outcome (“gate-out delay of leg L at cutoff C”)

4.1 Global traffic-tier assignment (NEW, binding — V3.9-f.7)

Problem fixed: §37.1 — all 4,053 universe-only airports provisionally REGIONAL with `traffic_prior=1.0` was not a stratification. Tier must be measured independently of Phase-6 outcomes.

Item	Candidate value (pending freeze at manifest — NOT frozen until reference data obtained)
Traffic measure	CANDIDATE: Annual scheduled departures — alternative annual operations / passenger movements listed as <i>pre-freeze option</i> , not mixed; final metric chosen once at freeze (see A30_3 #1)
Source	CANDIDATE sources: OAG/Cirium scheduled traffic or ACI/FAA/BTS annual operations — pick one at freeze; fallback <code>AeroDataBox proxy</code> only if no record, documented
Reference period	Single 12-month window ending 30 days before frame freeze (e.g. 2025-07-01 to 2026-06-30) — never a rolling window that moves during Phase 6
Thresholds	CANDIDATE rules: HUB: top ~7% OR 25,000 scheduled departures/year (whichever yields ~30); MID: next ~20% OR 5,000–25,000 — pick one deterministic rule at freeze (not both), record counts AND cut-values (<code>hub_cut</code>)

Item	Candidate value (pending freeze at manifest — NOT frozen until reference data obtained)
HUB definition	Satisfies HUB threshold AND feed-eligible
MID definition	Satisfies MID threshold AND not HUB
REGIONAL definition	Remainder of frame
Missing-reference policy	No traffic record → REGIONAL , tier_source='missing_reference' , traffic_prior=1.0 , flagged traffic_unverified ; never imputed as HUB
Tie/boundary policy	Equality at cut → higher tier; deterministic tie-break by ICAO lexical order
Versioning	After selection: traffic_source_name , traffic_source_version , traffic_retrieval_date , traffic_period , tier_version , tier_hash (SHA-256 sorted ICAO→tier) in adb_collection_meta — <i>not frozen until chosen</i>
Rebuild	After freezing, rebuild clean.adb_sampling_frame (tier, tier_source, traffic_prior) and regenerate 18-cell counts before Gate 1/2. Do NOT derive tiers from Phase-6 delay outcomes

traffic_prior for frame: HUB=3.0, MID=1.5, REGIONAL=1.0 (or normalized equivalent) — **CANDIDATE**, frozen with thresholds at manifest.

4.2 Macro-region mapping (NEW, binding — V3.9-f.7)

Replaces undocumented ICAO-first-letter heuristic (§37.2). Six **macro-regions** (the plan's priority anchor regions, §23):

1. **North America** (NA) — US, Canada, Mexico, Caribbean, **Central America north of 7°N** (ISO: US/CA/MX/PA/CR/etc.)
2. **Europe** (EU) — Europe + Turkey, **Russia west of 60°E (longitude override)** + Iceland/Greenland (B) — *east of 60°E* → AP

3. **Asia-Pacific** (AP) — East/South/Southeast Asia, **excluding Australia/NZ/Pacific islands** (those → OC)
4. **Gulf/Africa** (MEA) — Middle East, Africa (O,H,F,D,G)
5. **South America** (SA) — S South America **only** (ISO SA countries, **no Central America**)
6. **Oceania** (OC) — Australia, NZ, Pacific islands **only** (ISO AU/NZ/PG/FJ etc. or ICAO Y,N,A with country override) — **Every airport → exactly one region; country→region default, with explicit airport ICAO or longitude>60°E overrides for Russia/Turkey etc., recorded in region_overrides table**

Item	Frozen value
Source	ISO 3166 country → macro-region lookup (default, 1:1), primary key ISO country code; with explicit airport/longitude overrides (Russia split by 60°E, Turkey EU) in region_overrides — ICAO prefix only as final fallback when country unknown
Mapping table	country_code → macro_region 1:1 table, versioned (region_mapping_version, region_mapping_hash)
Exceptions	Explicit overrides table: Turkey LT→EU, Russia UU/UR west of 60°E→EU else AP, Greenland BG→NA, Australia Y→OC (not AP) — versioned in region_mapping + region_overrides hash
Unknown policy	No country after fallback → region='UNMAPPED', region_source='missing' , excluded from strata until manually classified; never silently assigned
Versioning	region_mapping_source, region_mapping_version, region_mapping_retrieval_date, region_mapping_hash in manifest

Item	Frozen value
Enforcement	<code>build_stratified_catalog.ts:macroRegionForIcao()</code> replaced by <code>country→region</code> default + <code>override</code> lookup (longitude check for Russia), fallback ICAO; 18-cell <code>tier×region</code> recomputed after rebuild, asserted <code>count(distinct regions)==1</code> per ICAO

4.3 Population scope (NEW, binding — V3.9-f.7, §37.3)

Frozen for the entire pipeline `population → capture → snapshots → evaluation` (same scope unless another population is separately named):

Category	Included in core experimental population?
Scheduled commercial passenger (operating flights with published schedule)	YES — core
Cargo (<code>withCargo</code>)	NO — excluded from core population; may be recorded as auxiliary <code>population_cargo</code> but not in primary denominator
Private / general aviation (<code>withPrivate</code>)	NO — excluded
Charter / non-scheduled	NO — excluded (no stable FIDS schedule)
Canceled (known canceled)	YES, counted in population — snapshot eligible, <code>outcome=canceled</code>
Diverted (actual destination scheduled)	YES, counted — snapshot eligible, <code>outcome=diverted</code> , <code>diversion_flag=true</code>
Codeshared marketing numbers	NO as separate population rows — deduplicated to operating leg (see §7.1 codeshare)

FIDS query (see §5.1) uses `withCargo=false`, `withPrivate=false`, `withCodeshared=true` (to capture operating leg + marketing links for dedup), then filter to scope above.

4.4 PRE vs POST airport eligibility (NEW, binding — V3.9-f.7, §37.4)

Decision (Option B — separate eligibility, explicit):

- **PRE (provider-observable prediction population, S1):** requires `pre_eligible=true` (`feed_schedule` available) — an airport can only contribute `flight_population` / `flight_snapshots` at T-24/T-6/T-90 if schedule feed exists for that horizon.
- **POST/AIRBORNE:** requires `post_eligible=true` (live OR adsb feed) — an airport can only contribute `raw_airborne_events` / `flight_airborne_snapshots` if live/ADS-B feed exists.
- **Core experimental batch slots:** the 4 daily slots are typed: HUB slot and the 2 MID slots require **both** `pre_eligible` && `post_eligible` (so a batch can always produce both PRE snapshots and POST trajectories); the REGIONAL slot requires `post_eligible=true` (and `pre_eligible` preferred but not required) — if no dual-eligible REGIONAL exists in a stratum, controller draws from `post_eligible`-only and marks `pre_eligible=false` → that flight's PRE snapshot marked `pre_ineligible`.
- PRE horizons never depend on an airport lacking required FIDS/schedule coverage — controller refuses to schedule a T-24 snapshot for an airport where FIDS withCargo/withPrivate scope cannot be validated.

POST-only REGIONAL policy (A31 §25): POST-only REGIONAL airports (`post_eligible=true`, `pre_eligible=false`) are collected as **auxiliary/exploratory data** and EXCLUDED from primary denominator-based POST claims. The primary POST denominator is `flight_population` (FIDS-observable, §5) intersected with airborne eligibility — this requires FIDS/schedule coverage, which POST-only airports lack. POST-only data is used for: (a) trajectory model training (supplementary), (b) coverage diagnostics, (c) marginal-value exploration. Do NOT use POST-only data for: (a) primary POST claims, (b) denominator-based metrics, (c) deployment decisions. Record `post_only_regional = true/false` on each batch and flag in evaluation metadata.

Controller (`pickAirportCandidates`) now filters: HUB/MID: WHERE `pre_eligible=true` AND `post_eligible=true` AND `in_frame=true`; REGIONAL: WHERE `post_eligible=true` AND `in_frame=true` (with `pre_eligible` preference flag).

4.5 Frame-balancing reference variables (NEW, binding — V3.9-f.7, §56)

Variable	Source	Formula / definition	Reference period	Missing policy	Manifest fields
Network degree	Scheduled route graph from external reference (OAG, historical FIDS archive, or airline schedule data — NOT from flight_populations which doesn't exist at frame- build time; A31 §9/§48)	Directed: out-degree = distinct destinations with 1 scheduled departure/week; in-degree similarly; undirected degree = distinct connected airports either direction ; report all three, primary = undirected	Same 12-month window as traffic tier	No sched- ule data → degree=NULL degree_unverified	degree_source, degree_period, degree_threshold if (week) degree_version

Variable	Source	Formula / definition	Reference period	Missing policy	Manifest fields
Carrier diversity	Same external reference schedule (NOT flight_population)	$\text{carrier_count} = \frac{\text{distinct operating carriers}}{\text{with 5\% of departures at airport; primary metric effective carriers}}$ $= 1/\text{HHI}$ where $\text{HHI} = \sum p_i^2$; alternative Shannon $H = -\sum p_i \ln p_i$ reported but not primary	Same 12-month	No data → NULL	carrier_diversity_source, carrier_period, carrier_metric (1/HHI), carrier_threshold
International/domestic mix	Same external reference (NOT flight_population)	$\text{intl_share} = \frac{\text{international scheduled departures}}{\text{total scheduled departures}}$ (route-level intl = origin country destination country)	Same	NULL	intl_source, intl_period

Variable	Source	Formula / definition	Reference period	Missing policy	Manifest fields
Exogenous traffic score	Same traffic measure as §4.1	$\text{traffic_score} = \min(1, \text{scheduled_departures} / \text{hub_cut})$ normalized to [0,1]	Same	NULL	as §4.1

Traffic score saturation (A31 §44): The $\min(1, \dots)$ clipping means all airports above `hub_cut` get `traffic_score = 1.0`, making the 40% traffic component useless for differentiation among large airports. This is INTENTIONAL — the traffic component is a coarse prior (HUB vs MID vs REGIONAL), not a fine-grained ranking. Differentiation among same-tier airports comes from the other 60% (geo, carrier, yield). If saturation is problematic, consider log-scaling: `traffic_score = min(1, log(1 + scheduled_departures) / log(1 + hub_cut))`. Measure at Gate 0.5 and decide. | **Geo/network diversity score** | Derived from degree + intl_share + macro-region dispersion | `geo_score = 0.5*degree_norm + 0.3*intl_share + 0.2*region_rarity` (where `region_rarity = 1 - region_share`) — frozen formula | Same | NULL | `geo_formula_version` | | **Carrier/international diversity score** | Carrier diversity + intl_share | `carrier_score = 0.6*effective_carriers_norm + 0.4*intl_share` — frozen | Same | NULL | `carrier_formula_version` |

Anchor normalization definitions (A31 §45): The formulas above use `degree_norm` and `effective_carriers_norm` but do not define the normalization. Freeze at manifest: - `degree_norm = min(1, degree / degree_cap)` where `degree_cap = 90th percentile of degree distribution across frame airports (measured at Gate 0.5)`. - `effective_carriers_norm = min(1, effective_carriers / carriers_cap)` where `carriers_cap = 90th percentile of effective carriers distribution`. - `intl_share` is already [0,1] (proportion of international departures). - `region_rarity = 1 - region_share` where `region_share = proportion of frame airports in the same macro-region`. - All normalization caps are frozen in manifest at Gate 0.5 and re-verified at FREEZE.

All balancing variables use the **fixed reference snapshot before frame freeze**, never the recursive current sample (§4).

4.6 Coverage-age core definition (NEW, binding — V3.9-f.7, §66)

coverage-age 5 d core means: airports whose `last_direct_observation_at` (from `adb_ingest_events` or `flight_events`) is within 5 days of the current batch's `window_start`. Membership is **recomputed per batch** (not frozen

globally), reported as `core_airports`, `core_share` in diagnostics. `core` is a diagnostic subset of the frame, not a separate frame.

5. The provider-observable prediction population (S1 — the census layer, renamed)

Why it exists: the webhook is an event stream, not a census. “These flights emitted an update we captured” “these are all the flights that existed at the cutoff.” Observability selects on airport size, activity, disruption severity, tracking quality, airline, region, ADS-B coverage — the very things we predict.

Naming (Gate-0 fix): the FIDS/schedule layer is called the “**provider-observable prediction population**”, NOT a “true census”. It is “population relative to the validated AeroDataBox-supported operational frame.” **Gate 5** validates provider-population → reference operational source (e.g., FAA/BTS-derived schedules/ops for US flights, where available) → observed.

Layer	Contents	Source
<code>flight_population</code>	one row per (flight, cutoff): “existed in the provider-observable prediction population at cutoff T”	AeroDataBox FIDS/schedule (2 API units per airport-window) for every collected airport+window
<code>flight_snapshots</code>	feature state at T−24 / T−6 / T−90, features cutoff	built for every population flight, not only event-captured ones; missing features marked missing, never dropped
<code>flight_outcomes</code>	five-state outcome (§7)	filled from later webhook events

Corrected snapshot rule (the ONLY normative rule): a snapshot exists iff the flight was in the provider-observable prediction population at cutoff eligible for the horizon. **Feature availability does NOT determine snapshot existence.** Each feature’s value enters the snapshot only if `information_available_timestamp` `prediction_cutoff`; unavailable optional features are NULL with `feature_missing=true`. A post-cutoff webhook event is required for the LABEL only — it never decides whether the snapshot exists. (This supersedes the old `V3_CollectionStrategy2.md` §11 line-667 rule, which was population-definition leakage.)

Coverage metrics (G5 + monthly): for every horizon, by airport tier / region / time-of-day / airline / tail-known: **population** → **captured** → **snapshot-eligible** → **snapshots** → **outcomes observed**.

Denominator funnels (A31 §67-68): The funnel above must be defined separately for PRE, AIRBORNE, and POST:

Funnel stage	PRE definition	AIRBORNE definition	POST definition
Population	Flights with <code>scheduled_gate_out</code> [cutoff, cutoff+window) in FIDS/schedule (S1)	Flights with airborne events in the observation window (S5)	Flights with POST-tier features (landing ETA, final delay)
Captured	Population flights that emitted a webhook notification in the window	Airborne flights with 1 raw airborne event	POST flights with 1 POST feature computed
Snapshot-eligible	Captured flights where <code>available_at</code> <code>prediction_cutoff</code> for all required features	Captured flights with trajectory reconstructable	POST flights with outcome observable
Snapshots	Snapshot-eligible flights with <code>flight_snapshots</code> row built	Airborne snapshots with trajectory features	POST snapshots with POST features
Outcomes observed	Snapshots where <code>outcome_state</code> = <code>observed</code> (not <code>active_censored</code> , <code>canceled</code> , <code>diverted</code> , <code>missing_outcome</code>)	Airborne snapshots where terminal event observed	POST snapshots where final delay observed

Each stage counts separately. Report `population_count`, `captured_count`, `snapshot_eligible_count`, `snapshot_count`, `outcome_observed_count` per horizon per airport. The denominator for any metric is the stage immediately above it (e.g., “outcome observed / snapshots” not “outcome observed / population”).

Full coverage taxonomy (restored — one airport belongs to exactly one state):

NOTE (A31 §49): The taxonomy above is NOT mutually exclusive as written. States like `zero_yield_once`, `zero_yield_repeated`, `zero_yield_persistent` can overlap with `directly_subscribed` and `recently_observed`. An airport can be `directly_subscribed` AND `zero_yield_once` at the same time. To make it mutually exclusive, define a priority order: 1. `coverage_failed` (highest — if feed/probe failed, nothing else matters) 2. `stale` (if no observation in >60 days) 3. `zero_yield_persistent` (if persistent empties) 4. `zero_yield_repeated` (if 2 empty observations) 5. `zero_yield_once` (if 1 empty observation) 6. `recently_observed` (if directly subscribed AND produced observations) 7. `directly_subscribed` (if subscribed but no observations yet) 8. `edge_discovered` (if observed via another airport) 9. `eligible` (if in frame but no subscription) 10. `supported` (if in AeroDataBox universe but not eligible)

An airport transitions states based on observation history. Record `coverage_state` and `coverage_state_changed_at` in diagnostics.

Plus per-frame **coverage-age distribution** (hours since last direct observation, by tier/region) reported monthly and at G5. The mapping **global frame vs directly observed vs edge-discovered** is kept explicit end-to-end so the GNN never conflates “not observed” with “no edge” (§12.3).

Population-claims rule: until the census layer is validated, all results are “under the collection regime”, never population-representative.

5.1 Complete FIDS protocol (NEW, binding — V3.9-f.7, §40)

Verified against AeroDataBox current docs (2026-08-28, OpenAPI v1.15.3.0, GET `/flights/airports/{codeType}/{code}/{fromLocal}/{toLocal}` endpoint, Tier-2) and required to be re-verified live at Gate 0.5. Provider contract repinned 2026-08-31 (A31 §7); every parameter frozen in `adb_collection_meta` (`fids_protocol_version`):

Parameter	Frozen value
Endpoint	GET /flights/airports/{codeType}/{code}/{fromLocal}/{toLocal} (NOT /flights/schedule) — version recorded at Gate 0 (<code>fids_api_version</code> , <code>fids_docs_date</code> , <code>openapi_sha256</code>)

Parameter	Frozen value
Direction	<code>direction=Arrival</code> or <code>direction=Departure</code> or <code>direction=Both</code> (single parameter, NOT <code>withDepartures/withArrivals</code>) — default <code>Both</code> gives full population; arrivals+departures needed for network/chain features
<code>withLeg</code>	<code>true</code> — includes information from the opposite movement and returns <code>departure + arrival</code> rather than only <code>movement</code> (A31 §8.4 — NOT “leg-detail mode”)
<code>withCancelled</code>	<code>true</code> — canceled flights ARE population members (§4.3); preserves <code>Canceled</code> , <code>Diverted</code> , and <code>CanceledUncertain</code> as distinct states (A31 §8.5)
<code>CodeshareStatus</code>	<code>IsOperator + IsCodeshared +</code> <code>Unknown</code> — <code>Unknown</code> is genuinely ambiguous; perfect operating-leg dedup cannot be claimed; create <code>codeshare_resolution_status =</code> <code>resolved_operator /</code> <code>resolved_marketing /</code> <code>ambiguous_unknown</code> (A31 §8.6)
Cargo/Private	<code>withCargo=false</code> , <code>withPrivate=false</code> — only if provider supports these filters; otherwise prove exclusion via alternative field (A31 §8.7)
Charter/non-scheduled	Scope may not be distinguishable from FIDS alone unless <code>FlightType</code> field is proven available (A31 §8.8)
<code>withLocation</code>	<code>false</code> for population calls (location not needed for membership; kept for webhook)
Local from/to	<code>fromLocal / toLocal</code> in airport local time (see §5.3); max permitted interval is plan-dependent — freeze <code>fids_max_range_for_actual_plan</code> from live account evidence (A31 §8.2)

Parameter	Frozen value
Edge inclusivity	[fromLocal , toLocal) — left-inclusive; NOT guaranteed by OpenAPI — live-verify boundary behavior at Gate 0.5 with adjacent-window overlap check (A31 §8.9)
Pagination/truncation	No pagination; if practical result limit observed (e.g. 500-record cap), characterize through live evidence and implement generic recursive split strategy — do NOT assume result truncated flag exists in provider contract (A31 §8.10)
2×2h segment treatment	For noncontiguous windows: freeze segment_1_start/end , gap , segment_2_start/end , same airport set?, same parent experiment day?, subscription lifecycle, FIDS service intervals, population membership, cost accounting — gap flights must NOT silently enter the exposed/captured denominator unless explicitly classified (A31 §50)
Retries/timeout	3 retries, 15s timeout, maxDeliveryRetries=0 for subscriptions (separate)
Schedule revisions	Capture scheduledTime , revisedTime , predictedTime per movement; preserve as-of versions (see §5.2); provider timestamps come from provider clocks , not airport/airline clocks (A31 §27-28)

Timestamp model — provider clocks (A31 §27-28): Preserve all provider clocks separately — do NOT collapse them:

Timestamp field	Source	Definition	Nullable?
provider_notification_generated_utc	Notification envelope timestampUtc	When the provider generated the notification	NO

Timestamp field	Source	Definition	Nullable?
<code>provider_state_updated_utc</code>	<code>provider.lastUpdatedUtc</code>	When the provider's internal state was last updated	NO
<code>location_reported_utc</code>	<code>location.reportedAtUtc</code>	When the location was reported (airborne events only)	YES — non-location state updates may not have this
<code>received_timestamp</code>	Our HTTP receipt	When we received the webhook delivery	NO
<code>available_at</code>	Our durable ingestion	When the information became usable in our system	NO

Do NOT automatically call `lastUpdatedUtc = provider_published_utc` without defining what “published” means. If retaining a generic `provider_published_utc` field, derive it explicitly from the appropriate source clock and preserve originals.

Raw event timestamp NULLability (A31 §28): Non-location state updates (e.g., status changes, schedule revisions) may not have `reportedAtUtc`. Therefore: (a) do NOT make a single location-style `event_timestamp` semantically mandatory for every event type, (b) define event-type-specific source timestamps and/or permit NULL where the provider supplies no real occurrence time, (c) never replace unknown occurrence time with receipt time without a source flag. Record `timestamp_source = 'provider_notification' / 'provider_state' / 'location_reported' / 'received' / 'derived'` on each event. | Cancellations/diversions | Canceled → `status=canceled`; Diverted → `actual destination scheduled with diversion_flag`; CanceledUncertain → distinct state, NOT conflated with Canceled (A31 §23) | | Retimes | New time creates new `service_date` handling — see `flight_instance_id` (§7.1) | | Population membership | One row per `flight_instance_id` per cutoff where `scheduled departure [cutoff, cutoff+window)` — see §5.2 | | Raw preservation | Persist raw JSON + SHA-256 + `response_hash` + `retrieval_utc` + `fromLocal/toLocal` + `timezone`; raw records are **immutable** — no UPDATE after initial persist (A31 §29-31) | | Retrieval timestamp | `fids_retrieval_utc` distinct from `event_timestamp` |

5.2 T–24 operational acquisition (NEW, binding — V3.9-f.7, §39)

It is **forbidden** to reconstruct T–24 from a final schedule observed later. T–24 is defined as **scheduled milestone T minus 24h** where T is

the `selected_t_milestone` frozen at Gate 0.5 from §6.0 (candidate: `scheduled_gate_out`; fallback: `scheduled_wheels_off`). Proven in two permitted ways:

Preferred — live schedule snapshots:

future airport/window assignment FROZEN early enough (25h before earliest T-24)

- actual FIDS observation at T-24 (live call, §5.1)
- actual FIDS observation at T-6
- actual FIDS observation at T-90m
- experimental webhook window (collection)
- later outcomes

Scheduler obligation: no airport/window may be assigned less than 25h before its earliest T-24 snapshot, so the T-24 FIDS call can be executed live. `assignment_frozen_at` recorded per batch. If assignment is too late, T-24 for that flight is marked `t24_unavailable` and excluded from primary.

Alternative — historical versioned as-known-at-time (only if provider preserves genuine versioned history): If used, must prove provider retains trustworthy versioned as-known-at-T state + valid `available_at` timestamps, and ETL uses `information_available_timestamp` T-24 to fetch the version observable at T-24. Proof required at Gate 0.5 before approval.

Code paths: `scripts/build_stratified_catalog.ts` (assignment), `server/lib/disruption/fidsCensus.ts` (new — FIDS fetcher, TBD until Gate 5), `historical_feature_store` as-of lookup.

5.3 FIDS timezone / DST handling (NEW, binding — V3.9-f.7, §41)

Provider FIDS `fromLocal/toLocal` bounds are **airport-local time**. Fixed `UTC±offset` is forbidden.

canonical UTC experimental interval (e.g. `window_start` UTC, cutoff UTC)

- airport IANA timezone (from airport metadata, e.g. `America/Los_Angeles` for KLAX)
- convert to local date/time with DST-aware library (e.g. `Intl` / `date-fns-tz``)
- correct `fromLocal/toLocal` strings (e.g. `"2026-08-30 08:00"`)
- provider query
- persist original `fromLocal/toLocal` + IANA timezone + UTC equivalents (both)
- persist conversion used

Mandatory DST test cases (in `tests/fidsTimezone.test.ts`): spring-forward gap, fall-back repeat hour, southern hemisphere, and UTC window crossing midnight local. Fail → controller refuses.

5.4 REST/FIDS worst-case budget proof (NEW, binding — V3.9-f.7, §42 — expanded A31 §7/§11/§40/§50)

Replaces 1,000. Formula (frozen in manifest, re-verified at Gate 0 with actual `units_per_call` and live plan evidence):

BASE = days × airports/day × horizons × calls/horizon × units/call
 = 31 × 4 × 3 × 1 × 2 = 744 API units (4 airports/day, 3 cutoffs, 1 Both/call, 2u/call)
 NOTE: each "1 call" may require split into 2×2h segments if window > fids_max_range;
 split calls counted as separate calls in ledger.

VALIDATION = Gate-5 spot-check: 10 sample windows × 3 horizons × 2u = 60

OUTCOME_REST = outcome-acquisition REST calls (after webhook window closes)
 = budget TBD at FREEZE; initially ~20u placeholder

HISTORY_BOOTSTRAP = historical store / as-of lookups
 = budget TBD at FREEZE; initially ~20u placeholder

DIAGNOSTICS + REFERENCE = coverage refresh free + traffic/region refs ~20u

PLANNED RETRY BUDGET = 75 units (10% contingency, allows 3 retries on ~12 calls) - ENFORCEABLE

Central API-unit ledger (A31 §11): Manifest records per-category hard caps:

FIDS_TOTAL_UNIT_BUDGET	= TBD at FREEZE (must cover BASE + DIAGNOSTICS)
FIDS_RETRY_UNIT_BUDGET	= 75 (enforced global counter)
VALIDATION_UNIT_BUDGET	= 60 (Gate-5 spot-check only)
OUTCOME_REST_UNIT_BUDGET	= TBD at FREEZE
HISTORY_BOOTSTRAP_UNIT_BUDGET	= TBD at FREEZE

Before issuing **any** billable REST request:

if projected_total_units > protected_rest_budget:
 REFUSE / DEFER

Never steal API units from the Phase-6 Alert-credit allocation silently. **Probe budget (A31 §40):** WSSS at ~331 items/hour for 2 hours 662 credits — this EXCEEDS the stated 500-credit/day cap. Resolve BEFORE live probing: (a) increase authorized probe cap, (b) shorten duration, (c) choose a different reference, (d) use stop-at-cap protocol and treat duration as censored, or (e) redesign probe budget while preserving scientific comparability. Do not call a cap-truncated probe “2.0h exact.”

WORST CASE (conservative) = BASE 744 + VALIDATION 60 + OUTCOME_REST 20 + HISTORY_BOOTSTRAP 20 + DIAGNOSTICS 20 = 864; **ENFORCEABLE MAX = 864 + min(actual retries, 75) = 939 (worst 939 if all 75 used, 1,000)** — re-verify at Gate 0 with actual per-call costs.

Enforcement (A30_3 #5): Manifest records all per-category budgets; controller fidsCensus_v3.ts enforces **global unit counter with per-category sub-counters** — once any category exhausted, remaining calls in that category stop/defer. Per-call 2u verified at Gate0; if provider raises cost, re-run proof

before Phase6.

6. Data pipeline & provenance (S2–S5)

#	Delta	Rule
S2	Raw events immutable	adb_ingest_events never deleted/edited; every delivery retains subscription_id , batch_id , received_at , provider notification timestamp (if any), HTTP metadata, raw payload + SHA-256 , parser version, schema version, number of items, upsert outcome
S3	Event log before current state	flight_events (one row per flight-item observation) feeds flight_state (latest via dedup) — the dedup table is an operational convenience, never the only research dataset
S4	Provenance invariant	“Never destructively overwrite research provenance.” Rebuilding state from the raw log must be possible at any time

6.0 Exact T milestone (NEW, binding — V3.9-f.8, §38 — CANDIDATE pending Gate-0.5 verification)

T in T-24 / T-6 / T-90 is **NOT** generic “scheduled departure”. **Candidate** T is the **scheduled_gate_out** milestone below, but it is **NOT** final until **Gate-0.5** verifies provider **scheduledTime** semantics (A30_3 #3).

Item	Frozen value
Exact milestone	CANDIDATE: scheduled_gate_out per §6.3 FAA-ASPM mapping — traveler-facing pushback; if Gate-0.5 finds scheduled_gate_out unverifiable, FALLBACK scheduled_wheels_off (documented in manifest, never changed after)
Provider field path	CANDIDATE: <code>flight.movement.scheduledTime[gateOut]</code> → scheduled_gate_out — path verified at Gate-0.5 (see §6.3 milestone table); frozen only after verification (<code>t_milestone_provider_path</code> in manifest)
Original vs revised	T = original scheduled_gate_out (first published schedule) — not revisedTime after retiming; retime after cutoff does not move T
Service-date handling	service_date = local date of scheduled_gate_out in origin airport timezone (see §5.3); flights crossing midnight keep original date
Retime policy	Deterministic (A30_3 #4): retime < 2h before scheduled_gate_out → same flight_instance_id (retain original T , revised_departure_utc mutable); 2h or service_date shift → new flight_instance_id with retime_parent_id link (see §7.1, must match)

Midnight rotations (A31 §65): When a flight crosses midnight (e.g., departs 23:00 UTC, arrives 01:00 UTC the next day), the following rules apply:

1. **Service date:** **service_date** = local date of **scheduled_gate_out** at origin airport. A flight departing at 23:30 SGT (15:30 UTC) on Aug 31 has **service_date** = 2026-08-31 even if it arrives at 01:00 SGT on Sep 1.
2. **Flight instance ID:** Uses **service_date** from the origin, not the destination. A flight departing IAD at 23:00 EDT (03:00 UTC+1) on Aug 31 arriving LHR at 11:00 BST on Sep 1 has **service_date** = 2026-08-31.
3. **Crossover windows:** If a crossover window spans midnight (e.g., 22:00-02:00 UTC),

flights are assigned to the window based on their `scheduled_gate_out` time, NOT their `scheduled_gate_in` time. A flight departing at 23:30 UTC is in the 22:00-02:00 window even if it arrives at 01:30 UTC the next day. 4. **Cutoff handling:** For T-24/T-6/T-90m cutoffs, use `scheduled_gate_out` time (not `scheduled_gate_in`). A flight with `scheduled_gate_out` = 23:30 UTC on Sep 1 has T-24 = Sep 1 23:30 - 24h = Aug 31 23:30 UTC. 5. Record `midnight_crossing` = true/false on each flight for audit. | Timezone | `scheduled_gate_out` stored as UTC; derived from airport-local `scheduledTime` + IANA timezone per §5.3 | | Stability | T never changes after `flight_population` row created; later FIDS revisions create new `available_at` versions but not new T |

T-24 means T minus 24h, T-6 minus 6h, T-90m minus 90 minutes, all computed from `selected_t_milestone` UTC (frozen at Gate 0.5 — either `scheduled_gate_out` or `scheduled_wheels_off` fallback).

6.1 Dual prediction-state data contract (PRE + AIRBORNE) — NEW (V3.9-f.2)

The plan has **two distinct prediction states**, never merged into one row. **An event is NOT one of them.** `prediction_state` is a property of the **snapshot / training example**, derived at dataset-construction time; the raw event carries only immutable source facts (`event_phase`, `event_timestamp`, `data_stage` PRE|POST). The same observation (e.g. an event at 14:10) may feed features for multiple tasks → we never burn an event’s reusability by stamping it with one state. The two derived states:

<code>prediction_state</code> = PRE_DEPARTURE	cutoffs: T-24h / T-6h / T-90m valid while <code>prediction_cutoff</code> < actual <code>wheels_off</code> target: gate-out delay / arrival disruption / cancellation / diversion
<code>prediction_state</code> = AIRBORNE	cutoff: the provider observation timestamp <code>t</code> valid while actual <code>wheels_off</code> <code>t</code> < actual <code>wheels_on</code> (or terminal censoring, §6.2) target: landing ETA, remaining flight time, wheels-on delay / gate-in delay, P(delay>15)

The cutoff rule is universal: for BOTH states, `feature/provider_timestamp prediction_cutoff`. For AIRBORNE the cutoff is the flight’s own observation timestamp (e.g. `reportedAtUtc`), so the model may use actual departure time (known at `t`) but never actual arrival time (that is the target). This generalizes the pre-departure rule from `scheduled departure` to `prediction timestamp`.

Availability-time rule (NEW, blocking — the deepest leakage fix): “the event happened before the cutoff” is NOT sufficient. A model predicts from what the **system could actually know at the decision time**, so the

eligible condition is:

`feature eligible iff information_available_timestamp prediction_cutoff`

where `information_available_timestamp` = when OUR system had the value (the time a downstream feature could actually be built from it). This is stricter than `event_timestamp prediction_cutoff`, which only proves the thing happened, not that we knew it. Every preserved observation carries up to four distinct timestamps and they are NEVER conflated:

`event_timestamp` when the thing actually happened/departed/etc.
`provider_published_utc` when the provider generated/published the observation
`available_at` when OUR ingestion made it usable for feature-building
`received_timestamp_utc` when our system received the raw delivery

Example that must NOT leak: `reportedAtUtc = 14:00, provider_published = 14:01, available_at = 14:07`. With a prediction cutoff of 14:05 the observation is INELIGIBLE even though `reportedAtUtc = 14:05`, because the information was not in the system until 14:07. This discipline applies uniformly to: flight events, FIDS/schedule revisions, live flight status, weather observations & forecasts (TAF amendments), network/congestion features, and tail/rotation features (previous-leg arrival must be both landed AND its arrival value available, by cutoff). `available_at` is computed at the ETL layer from the S2 raw ledger (received timing), never backfilled from the provider or from a later dataset.

6.2 Airborne data foundation (NEW — what the collector must preserve)

AeroDataBox already provides the fields — verified in our own code: `server/lib/disruption/flightStatus_v3.ts` defines `liveLocation` with `lat`, `lon`, `altitude`, `pressureAltitude`, `groundSpeed`, `trueTrack`, `vsiFpm`, `reportedAtUtc`, and `flightNotificationExtractor_v3.ts` flattens all of them (`loc_lat`, `loc_lon`, `loc_altitude_ft`, `loc_pressure_altitude_ft`, `loc_ground_speed_kt`, `loc_true_track_deg`, `loc_vsi_fpm`, `loc_reported_utc`, `has_live_location`, `data_stage` PRE|POST). The missing piece is a **preservation + time-series contract**, not new provider data.

Hard requirement: do NOT reduce the webhook payload to “latest location” before storing. `flight_events` must keep one row per observation so the trajectory is reconstructable. Current code DANGER (verified `flightDataPrePostStore_v3.ts`): the upsert dedup key is `SHA-256(flight|carrier|lastUpdatedUtc)` — if the provider updates location under the same `lastUpdatedUtc`, earlier points are overwritten. **S5 fixes this** (§15): the research event log keys on (`flight`, `carrier`, `locReportedUtc`) (plus fallback) so every airborne observation is preserved; the dedup `flight_state` table remains the latest-state convenience and is never the trajectory source.

Per-observation fields to preserve (raw collection layer):

Group	Fields
Identity	flight_number, callsign, carrier, aircraft_reg, aircraft_mode_s, aircraft_model, icao24 where available
Times	event_timestamp (reportedAtUtc), provider_published_utc, available_at, received_timestamp_utc (four distinct timestamps per §6.1 availability rule — never conflated), source_latency_seconds (received — event)
Scheduled/revised	dep_scheduled_utc, dep_revised_utc, arr_scheduled_utc, arr_revised_utc, original/current/actual destination
Movement chain	FAA-ASPM-style milestone set (8 fields, scheduled AND actual, never conflated): scheduled_gate_out, actual_gate_out, scheduled_wheels_off, actual_wheels_off, scheduled_wheels_on, actual_wheels_on, scheduled_gate_in, actual_gate_in (mapped from dep_scheduled_utc, dep_revised_utc, dep_runway_utc, arr_runway_utc, arr_scheduled_utc, arr_revised_utc, actual_gate_in; each field keeps its own available_at per §6.1). Gate/taxi/airborne/block delays are all derivable and never conflated
Airborne state	lat, lon, altitude_ft, pressure_altitude_ft, pressure_hpa, ground_speed_kt, true_track_deg, vsi_fpm, on_ground, flight_phase

Group	Fields
Derived (stored raw, computed in ETL)	distance_to_destination, distance_flown, fraction_of_route_completed
Context	weather_snapshot_id, eta_provider, eta_model_reference
QC	data_quality_flag, trajectory_gap_seconds, source_latency_seconds

Milestone-mapping verification (NEW, required before schema freeze — do NOT blindly rename provider fields): AeroDataBox exposes `FlightAirportMovementContract` with `scheduledTime`, `revisedTime`, `predictedTime`, `runwayTime`, `gate`, and other movement fields. That DOES NOT automatically mean every provider `scheduledTime` equals FAA’s `scheduled_gate_out` or `scheduled_wheels_off`. Before the milestone schema is frozen (and re-confirmed in the Gate 0.5 sample inspection), verify the ACTUAL provider JSON semantics for each of the eight §6.2 milestones: `mapped_<milestone> = <provider field path> + <semantics note>` (which milestone it counts as and any caveat), recorded in the code/config, so a provider `scheduledTime` is never renamed `scheduled_gate_out` unless the provider’s field semantics support that exact interpretation. Confirm `runwayTime` means wheels-off/wheels-on (it does, per provider docs) and that gate-out/gate-in have a real source before labeling them. This milestone mapping is a binding §6.2 requirement; Gate 0.5 re-verifies it on live payloads (§17 step 16). If a milestone cannot be verified (e.g. no reliable gate-in source), it is kept NULL and explicitly marked `milestone_unverified`, never approximated.

Flight phase (explicit): `pre_departure`, `taxi_out`, `airborne_climb`, `airborne_cruise`, `airborne_descent`, `approach`, `landed`, `taxi_in`, `gate_in` — so ETA at 10 min after takeoff is never pooled with ETA 100 NM from destination.

Trajectory QC / provenance (S5): never overwrite raw points; pipeline is `raw_airborne_events` → `clean_airborne_points` → `flight_trajectory` → `flight_airborne_snapshots` → Model POST. Cleaning removes impossible lat/lon/alt/speed, sorts timestamps, joins flight identifiers, marks unjoinable points — mirroring the Springer 2024 in-flight-ETA study (§19).

Observation cadence policy: do NOT assume a fixed cadence (the provider sends on update, not on a guarantee). Define and measure: `post_departure_observation_policy` with target cadence, minimum acceptable, and max gap; report `obs_per_flight`, `median_gap_seconds`, `P95_gap_seconds`, `max_gap_seconds`, `trajectory_duration`, `trajectory_completeness_`%. Measure in the canary (Gate 0.5) and monthly. **Only if measured cadence**

is insufficient do we consider supplementary REST — never before measurement.

Terminalization rule (when a flight is “complete”): declare final `flight_outcome` at the FIRST of: landing/gate-in, diversion terminal state, cancellation, provider terminal state, or **censoring after a documented grace interval**. The ETL must never finalize a label before the terminal event arrives.

Disruption event definition (A31 §69): A “disruption event” is defined as: 1. **Cancellation:** `status = Canceled` or `CanceledUncertain` in provider FIDS/webhook data. 2. **Diversion:** `actual_destination` `scheduled_destination` (provider-reported or derived from flight trajectory). 3. **Severe delay:** `actual_wheels_off - scheduled_wheels_off > 60 minutes` (departure delay) OR `actual_wheels_on - scheduled_wheels_on > 60 minutes` (arrival delay). 4. **MCD (Major Command Decision):** ATC ground stop, ground delay program, or airspace flow program affecting the airport (if available from FAA/ATC data sources). 5. **Weather disruption:** METAR/TAF reports visibility < 1 mile, ceiling < 500 ft, or wind > 30 knots (if available).

Each disruption event gets a `disruption_event_id` (deterministic, seeded) and is used for: - Engine E partitioning (whole disruption event in one partition, §13.4) - Event-level sampling (`sampling_strategy='event'`, Month 2+)

Event-regime definition (A31 §71): The “event-regime” is a sampling strategy where disruption events (§69) are oversampled relative to their natural frequency. Define: 1. **Event definition:** A disruption event is any flight with `disruption_event_id` NOT NULL (§69). 2. **Event frequency:** Measure natural disruption frequency at Gate 0.5 (e.g., ~5% of flights are disrupted). Record `natural_event_frequency` in manifest. 3. **Event sampling rate:** Month 2+ only. Sample event flights at `event_sampling_rate = 2×` (or configurable) relative to natural frequency. Non-event flights sampled at `1×`. 4. **Event-regime partition:** Engine E uses whole disruption events in one partition (§13.4). Event-regime robustness test: train on `train-standard` → test on `test-event-regime` (regime as blocking/ablation variable). 5. **Event-regime metadata:** Record `sampling_strategy = 'event'`, `event_sampling_rate`, `event_regime_start_date` on each batch. 6. **Month 1:** Event-sampling is OFF. All flights sampled at natural frequency. Event-regime analysis is exploratory only. - Disruption stress testing (§13.4)

Record `disruption_event_source` (provider/ATC/weather) and `disruption_event_timestamp` on each event.

Censoring timeout/grace (NEW, tightening): “our collection window ended” “the flight never produced an outcome.” A flight is only labeled `active_censored` / `missing_outcome` AFTER a separately-defined grace interval past the last observation (or past the window end) during which a terminal event could still arrive:

last observation / window end

wait grace interval (defined once at freeze; proposal: 60 min, or
provider-typical arrival-notification latency + margin, measured at Gate 0.5)

terminal event arrived? → label it normally
no terminal event? → active_censored / missing_outcome

Until that grace interval elapses the label is NOT final; the ETL never finalizes a label early. `missing_outcome` = grace elapsed with no terminal event. The grace value is frozen with the manifest (§17 Phase 5), and the canary measures actual arrival-notification latency to justify it.

Airborne snapshots table (S5):

```
flight_airborne_snapshots:
flight_id | airborne_snapshot_id | prediction_state='AIRBORNE'
event_timestamp | provider_published_utc | available_at | received_timestamp_utc
icao24 | registration | callsign | flight_number | airline | aircraft_type
origin | destination | scheduled_departure | scheduled_arrival
scheduled_gate_out | actual_gate_out | scheduled_wheels_off | actual_wheels_off
scheduled_wheels_on | actual_wheels_on | scheduled_gate_in | actual_gate_in
latitude | longitude | altitude | ground_speed | heading | vertical_rate | on_ground | flt
weather_snapshot_id | weather_timestamp_utc
distance_to_destination | distance_flown | fraction_of_route_completed
eta_provider | eta_model_reference | current_operational_destination
data_quality_flag | trajectory_gap_seconds | source_latency_seconds
```

POST labels — targets are milestone-explicit (NEW, blocking): “scheduled departure/arrival” is ambiguous, so every label names its exact milestones (FAA-ASPM definitions, §6.2). Two delay families and one ETA family, all collected: - **POST-A — ETA / remaining flight time:** `label_eta_landing = actual_wheels_on - t` - **POST-B1 — landing delay (wheels-on):** `label_arr_delay_wheels_on = actual_wheels_on - scheduled_wheels_on` - **POST-B2 — gate-arrival delay (traveler-facing):** `label_arr_delay_gate_in = actual_gate_in - scheduled_gate_in` Evaluated separately (SJSU airborne-ETA work supports the ETA target; §19); `label_arr_delay` alone is deprecated because it does not say which milestone it means. PRE-side targets are equally explicit: `departure delay = actual_gate_out - scheduled_gate_out` (gate, traveler-facing) AND/OR `actual_wheels_off - scheduled_wheels_off` (airborne/trajectory), stored as separate columns, never merged into one ambiguous “dep delay”.

6.3 Eight OOOI/ASPM milestone mapping (binding — V3.9-f.7, §47)

Each of the 8 milestones maps one FAA-ASPM concept to one AeroDataBox JSON path, verified before freeze and re-verified at Gate 0.5. Never invent gate time from runway time.

#	Milestone	Provider JSON path (example)	Provider description	Semantic interpretation	FAA/ASPM equivalence	Null if unverified	Availability timestamp
1	scheduled_departure	departure.scheduledTime (verify)	push-out	Gate-out scheduled	scheduled_time_per ASPM	highest confirmed	available_at = true of that movement
2	actual_gate_movement	gate.actualTime if gate source exists	push-back	Gate-out actual	actual_gate_out	if no gate source (never derive from runway)	available_at
3	scheduled_wheels_off	departure.scheduledTime	wheels-off	scheduled	scheduled_time	if unverified	available_at
4	actual_wheels_off	departure.actualTime	wheels-off	actual	actual_wheels_off	be verifiable (provider confirms runway-Time = wheels-off)	available_at
5	scheduled_wheels_on	arrival.scheduledTime	wheels-on	scheduled	scheduled_time	if unverified	available_at

#	Milestone	Provider JSON path (exam- ple)	Provider descrip- tion	Semantic inter- preta- tion	FAA/ASPN equiva- lence	Null if unveri- fied	Availability times- tamp
6	actual_wheels_on	actual_wheels_on	arrival wheels-on	Runway actual	actual_wheels_on	be veri- fiable	available_at
7	scheduled_gate_in	scheduled_gate_in	or arrival	Scheduled gate in	scheduled_gate_in	Gate_in	inavailable_at
8	actual_gate_in	actual_gate_in	gate arrival	Actual gate in	actual_gate_in	if no gate source	available_at

For each: `provider_description` = docs quote, `caveat` = any mismatch, `verification_source` = code `flightNotificationExtractor_v3.ts:xxx` + Gate 0.5 payload evidence. If unverifiable → keep NULL + `milestone_unverified=true`, never approximate. Gate 0.5 records live JSON examples for all 8.

6.4 Four-timestamp provenance (binding — V3.9-f.7, §50)

Every observation preserves separately:

Timestamp	Source	Nullable?	Ordering vs prediction_cut-off	Off rule
event_timestamp	Provider reportedAtUtc / locReportedUtc — when thing happened	NO	event_timestamp	If missing → observation prediction_cut-off necessary but NOT sufficient
provider_published_timestamp	Provider lastUpdatedUtc — when provider published	YES (NULL if not provided)	Must be available_at if both present	Negative latency (published > received) → quarantine
received_timestamp	adb_ingest_events.received_at	NO	Always available_at	Clock skew >5 min → flag

Timestamp	Source	Nullable?	Ordering vs prediction_cutoff	Rule
<code>available_at</code>	ETL computes as <code>received_timestamp_utc</code> + processing lag (or when feature row becomes queryable)	NO (computed)	Eligibility: available_at prediction_cutoff (§6.1 universal rule)	If NULL → ineligible; duplicates earliest

Negative latency, `available_at < event_timestamp`, or `received < published - 5 min` → quarantine table, not silently corrected.

6.5 Snapshot provenance (binding — V3.9-f.7, §51)

Every `flight_snapshots` and `flight_airborne_snapshots` row carries deterministic provenance:

```
snapshot_provenance = { flight_population_row_id/hash, fids_response_hash,
schedule_version, weather_snapshot_id, historical_feature_row_ids
(array), feature_builder_version (git SHA), snapshot_builder_version
}
```

A researcher can answer “why was this feature available at this cutoff?” by joining `historical_feature_store` rows with `available_at cutoff`. Provenance stored as `provenance_json` or `provenance_hash` column.

6.6 Gate-0.5 measure→freeze items (binding — V3.9-f.7, §52)

Do NOT guess these — measure in canary/Gate 0.5, then freeze in manifest before Phase 6:

Item	How measured at Gate 0.5	Frozen in manifest
Censoring grace	Distribution of <code>arrival-notification</code> latency (<code>received - wheels_on</code>) median/P95/max + 10 min margin	<code>censoring_grace_minutes</code> (proposal 60)
<code>airborne_usable</code> min points	<code>obs_per_flight</code> distribution	<code>airborne_usable_min_points</code> (proposal 2 for line, 5 for curve)
Target cadence	<code>median_gap_seconds</code>	<code>target_cadence_seconds</code>

Item	How measured at Gate 0.5	Frozen in manifest
Minimum acceptable cadence	P95 gap	<code>min_acceptable_cadence_seconds</code>
Maximum gap	max	<code>max_gap_seconds</code>
Minimum trajectory duration/coverage	<code>trajectory_duration</code> + <code>completeness_%</code>	<code>min_trajectory_duration_seconds</code> , <code>completeness_threshold_%</code> , <code>completeness_formula</code>
Warning/fail rule	Thresholds breach → WARN vs FAIL	<code>cadence_warning_rule</code> , <code>cadence_fail_rule</code>

Gate 0.5 records sample size, all distributions, latency, and rationale paragraph before freeze.

7. Flight-outcome states & modeling populations (replaces “null = censored”)

State	Definition	Modeling use
<code>observed</code>	Target-specific label observed (see §7.3)	regression labels for that target only
<code>active_censored</code>	In window, label not yet observed, not canceled, grace not yet elapsed (see §7.4)	survival/censored tasks; excluded from regression labels for that target
<code>canceled</code>	status/payload indicates cancellation	cancellation classifier
<code>diverted</code>	<code>actual_destination</code> <code>original_scheduled_destination</code> AND reliable evidence (<code>diversion_flag</code>)	diversion classifier (rare, high volume)
<code>missing_outcome</code>	Grace elapsed with no terminal event	explicit “unknown”; kept out of label population; tracked as data-quality

- Per-flight retained fields: `original_scheduled_destination`, `current_operational_destination`, `actual_destination`, `diversion_flag`, plus `flight_instance_id` (see §7.1).
- Per-task label populations are **target-specific** (see §7.3): e.g. `wheels_on` delay regression = only snapshots where `wheels_on_label_observed=true`.

7.1 Canonical `flight_instance_id` (NEW, binding — V3.9-f.7, §43)

One **physical operated flight leg** = one prediction unit, not one marketing number.

Component	Source	Role
Preferred: provider <code>flightId</code> / <code>id</code> if stable across retimes/codeshares	AeroDataBox <code>flight.id</code> field, tested at Gate 0.5 for stability (same <code>id</code> across <code>withCodeshared</code> variants = stable)	<code>flight_instance_id</code> if verified stable; fallback otherwise
Fallback canonical key (frozen): <code>operating_carrier</code> + <code>operating_flight_number</code> + origin ICAO + destination ICAO (original scheduled) + <code>service_date</code> + <code>scheduled_gate_out_utc</code> <code>callsign</code>	Derived from FIDS/webhook <code>operating_carrier</code> , <code>flightNumber</code> , origin/destination, <code>scheduled_gate_out</code> per §6.0	Stable identity
<code>provider_record_key</code>	Where available	Disambiguation, not primary
<code>codeshare_handling</code>	Original FIDS record key	Link to raw FIDS row
	Marketing numbers → <code>marketing_flight_numbers</code> array on the operating leg; never create separate <code>flight_instance_id</code> per marketing number (see §7.2)	Prevents population inflation
Retime handling	Deterministic (A30_3 #4, must match §6.0): <code>scheduled_gate_out</code> change <2h → same <code>flight_instance_id</code> (mutable <code>revised_departure_utc</code>); 2h OR <code>service_date</code> shift → new <code>flight_instance_id</code> with <code>retime_parent_id</code> link	Identity vs state

Component	Source	Role
Diversion handling	Destination change → same <code>flight_instance_id</code> retains <code>original_scheduled_destination</code> ; <code>current_operational_destination</code> and <code>actual_destination</code> updated; <code>diversion_flag</code> set	See §7.4
Collision fallback	If fallback key collides (same carrier/number/origin/destination/date but distinct provider records) → append <code>provider_record_key</code> hash suffix	Uniqueness
Separation	<code>flight_instance_id</code> (stable) vs mutable <code>flight_state</code> (times, destinations, status) — never conflate	ETL principle

Code: `flightDataPrePostStore_v3.ts:researchEventKey()` extended to `flight_instance_id`; `fidsCensus.ts` canonicalizer; test `flightInstanceCanonical.test.ts`.

7.2 Codeshare deduplication (NEW, binding — V3.9-f.7, §44)

- `AeroDataBox` `withCodeshared` can return inferred codeshares with **false results possible** (docs). Prefer enough raw information to canonicalize internally to operating leg.
- **Marketing identifiers stored as attributes**, never as separate physical units. Counting distinct `flight_instance_id` (operating leg) gives correct `population`, `unique_flights`, `route counts`, `test partitions`.
- Verification at Gate 0.5: compare `withCodeshared=true` vs `false` payloads for same airport-window; confirm deduplication does not over-merge distinct operating flights.

7.3 Target-specific label observability (NEW, binding — V3.9-f.7, §48)

Generic `observed` flag retired. Each milestone has its own label-observed boolean:

Column	Meaning	When true
gate_out_label_observed	actual_gate_out known	Gate-out actual received and available_at grace cutoff
wheels_off_label_observed	actual_wheels_off known	Wheels-off actual received
wheels_on_label_observed	actual_wheels_on known	Wheels-on actual received
gate_in_label_observed	actual_gate_in known	Gate-in actual received

A flight may be wheels_on_label_observed=true but gate_in_label_observed=false (gate time missing). Each regression target uses **its own observed population**: e.g. gate-arrival delay model trains only on gate_in_label_observed=true rows. Never train gate-in delay on wheels-on population.

7.4 Censoring with frozen grace (NEW, binding — V3.9-f.7, §49)

Binding sequence (patches all stale “window ended = missing” wording):

collection/window ends

- outcome NOT automatically declared missing
- wait frozen grace interval (from §6.6, manifest)
- check task-specific terminal evidence (see §7.3)
- if any label_observed true → label normally (observed)
- else if canceled/diverted evidence → canceled/diverted
- otherwise → active_censored (if still within provider FIDS horizon) / missing_outcome (g

Grace measured at Gate 0.5 as arrival-notification latency (received - wheels_on) P95 + margin; frozen before Phase 6.

Outcome acquisition after webhook window (A31 §24): After the experimental webhook window closes, terminal events (wheels_on, gate_in) are acquired via: (a) FIDS REST calls at later horizons (T-6, T-24, T-90m per §5.1/§5.2); (b) webhook notifications that arrive after the window closes (late-arriving events are still valid if event_timestamp is within the window); (c) historical FIDS as-of lookups if provider preserves versioned history. For Flight Alert subscriptions, outcomes are acquired only while the subscription is active — if the alert window expires before the terminal event, the outcome is acquired via FIDS REST or marked missing_outcome. Do NOT assume webhook captures all terminal events.

7.5 Route / OD and aircraft/tail identity (NEW, binding — V3.9-f.7, §45-46)

Route/OD identity (§45): directed OD = (origin ICAO, original_scheduled_destination ICAO) — ordered pair, directed. original_scheduled_destination vs

operational_destination vs actual_destination kept separately; diversion represented as actual original + diversion_flag. Codeshare: route uses operating carrier OD, not marketing. Retime: route unchanged if destination same. Never silently convert directed to undirected.

Aircraft/tail identity (§46): Priority fallback: aircraft_reg (registration, e.g. 9V-SKD) > aircraft_mode_s > icao24 > provider aircraft ID. tail_known = (registration NOT NULL). missing_tail → chain break, not linked. conflicting identifiers (reg vs mode_s disagree) → tail_conflict_flag=true, not linked. aircraft_swap (tail changes mid-rotation) → chain break at swap. tail_chain_links counted only on consistent registration across legs within max_turnaround=6h and same calendar date.

AIRBORNE / POST population denominator (NEW, blocking — the POST-side census, mirroring §5): “flights with airborne events we happened to observe” is NOT a defined population. The **airborne_eligible denominator is population-defined independent of airborne observation capture** — it comes from the SAME flight_population layer as PRE (§5: provider-observable FIDS population), intersected with movement evidence that the flight actually became airborne (actual_wheels_off / provider terminal state). A flight is **airborne_eligible** whether or not a single airborne point was ever captured — using the airborne webhook to define the denominator would make it circular with airborne_observed. Every eligible flight then goes through the same measured funnel as PRE does, and each stage is counted and reported:

airborne_eligible	= flight_population (FIDS-observable, §5) actual_wheels_off (or provider terminal state) [denominator defined from FIDS/population evidence, NOT from whether we captured airborne points]
airborne_observed	1 airborne observation captured for that flight
airborne_usable	N usable points (N frozen in manifest; proposal 2 to fit a line, 5 to fit a curve)
airborne_trajectory_complete	passes trajectory-completeness threshold (completeness_% / max_gap_seconds, §6.2 QC)
POST_snapshot_eligible	produced 1 POST training snapshot (cutoff rule §6.1)
POST_labeled	received a final terminal label (§6.2 terminalization)

- **Report the funnel at every monthly checkpoint:** eligible → observed → usable → complete → snapshot_eligible → labeled, e.g. as airborne_eligible → airborne_observed → ... → POST_labeled proportions and per-airport / per-tier rows, exactly like the PRE census (§5).
- **Missingness is measured, not hidden:** airborne_eligible - airborne_observed quantifies provider coverage; observed - usable quantifies point quality; complete - snapshot_eligible quantifies cutoff/cadence losses. If a big step collapses (e.g. most flights observed

but few complete), that’s a provider-cadence finding (Gate 0.5 / monthly), and POST training is weighted to the funnel denominators — a POST model must NOT be silently trained only on the easy-to-track tail of flights (**airborne_observed** disproportionately on provider-friendly aircraft/routes).

8. Sampling design (LOCKED)

Element	Decision
Default window	1 × 4 h continuous per day (preserves aircraft-chain continuity)
UTC schedule	Run-level constrained randomized allocation : seeded balanced permutation of {00,04,08,12,16,20}; every 6-day block uses each UTC slot exactly once (HARD); minimize weekday×UTC imbalance $\Sigma(n_c - \bar{n})^2$ among valid permutations (SOFT). Seeded + replayable (time_window_schedule_seed). Never claim “balanced at the weekday×UTC level”
Calendar	26 × 4h + 3 × 2×2h + 2 × 6h = 31 days (84% / 10% / 6%)
Crossover (R6)	Template frozen BEFORE treatment: freeze candidate pool → airport set → UTC slot + day/block → crossover block → randomize window_shape → execute. Treatment must NOT depend on any post-freeze observation
Environmental context (restored)	every batch/window records weather severity (METAR/TAF-derived, §10), ATC delay-program flags , and storm-track context in batch metadata. The crossover does not make the operating environment identical — two matched windows can differ because severe weather arrived during one; that context must be explainable, not absorbed as “treatment effect”

ATC/storm/MCD context (A31 §70): Each batch/window must record: 1. **Weather severity:** `weather_severity_index` = composite of METAR/TAF conditions (visibility, ceiling, wind, precipitation). Scale: 0 (clear) to 5 (severe). Derived from §10 weather features. 2. **ATC delay-program flags:** `ground_stop` = true/false, `ground_delay_program` = true/false, `airspace_flow_program` = true/false. Source: FAA ATCSCC (if available); otherwise NULL. Record `atc_source` and `atc_timestamp`. 3. **Storm-track context:** `storm_track_distance_km` = distance from airport to nearest storm cell (if available from NOAA radar). `storm_track_direction` = direction of storm movement. Source: NOAA (if available); otherwise NULL. Record `storm_source` and `storm_timestamp`. 4. **MCD (Major Command Decision):** `mcd_flag` = true/false if any ATC/airspace program is active. Derived from ATC flags above.

These context variables are used for: - Crossover explanation (why two matched windows differ) - Environmental covariates in model features - Disruption event classification (§69) - Batch metadata audit trail

Record `environmental_context_json` on each batch row for full auditability. | Anchor pool | 5 airports; provisional KLAX/EGLL/WSSS/SBGR/OMDB, **finalized only after probing** (§9); 1/day, no-repeat-until-all, order randomized via `anchor_pool_seed` | | Anchor score | **40% exogenous traffic + 20% geo/network diversity + 20% carrier/international diversity + 20% standardized observed yield**; station/API capacity = separate feasibility GATE, not a score component; formula frozen in code pre-probe | | Yield metric | `yield_score` = $f(\text{unique_flights}/\text{credit}, \text{tail_chain_links}/\text{credit}, \text{stability})$, each standardized to [0,1], formula frozen pre-probe | | REGIONAL | normalized yield-aware draw (m_i [0.25,1.5], hard cap $\times 1.5$, $\Sigma p=1$); `airport_layer_design_probability` = p_i = conditional design probability at the draw; **adaptive rule is efficiency-oriented allocation, not representation-preserving**; boots only after probe data; before that uniform $1/|\text{eligible}|$ | | Long-tail | named “**coverage floor**” (efficiency allocation, not representation) | | `sampling_weight` | **NULL by default; no auto 1/p** (1/p valid flight-level inclusion probability) |

8.1 Scheduler contract (code must REFUSE)

- Experiment day without a declared template/crossover group → REFUSED.
- Template/experiment mismatch (different tier mix or slots) → REFUSED.
- Crossover period-2 without its period-1 → REFUSED.
- `requested_window_hours=6` hitting the cap → tagged up-to-6h, `stop_reason='budget_reached'`, never relabeled “6h”.
- Hard constraints (daily credit 1900, one batch/day, valid window/tier, crossover integrity, no duplicate anchor in cycle) are never sacrificed for the soft balance objective.

8.2 REGIONAL adaptation full specification (NEW, binding — V3.9-f.7, §53)

Retain m_i $[0.25, 1.5]$ and hard cap $\times 1.5$, but freeze the recurrence:

Item	Frozen value
Yield measure for adaptation	<code>yield_score_i</code> from last completed observation of <code>i</code> (from §9 yield formula); if never observed $\rightarrow$ NULL $\rightarrow$ use uniform
History window	Exponential moving average $\text{ema_yield_i}(t) = \alpha \cdot \text{yield_i}(t) + (1 - \alpha) \cdot \text{ema_yield_i}(t-1)$ with $\alpha = 0.5$, window ~ 4 most recent observations; frozen
Smoothing / cap	$\text{raw_m} = \text{ema_yield_i} / \text{median_ema_yield_frame}$; then $m_i = \text{clamp}(\text{raw_m}, 0.25, 1.5)$ and $\text{score_i} = \text{traffic_prior} \cdot m_i$ capped score_i $\text{traffic_prior} \cdot 1.5$ (no runaway) — <code>r_i</code> REMOVED (A30_3 #6, was undefined)
Update frequency	After each completed airport observation (not daily tick); m_i version incremented
Cold start	$m_i = 1.0$ until <code>ema_yield_i</code> exists (3 observations)
Missing data	No yield history $\rightarrow m_i = 1.0$ (not penalized)
Zero-yield response	See §8.3 — only repeated/persistent state may down-weight; once never down-weights
Reset	m_i never reset except manual adjudication entry; logs <code>m_i_history</code> array
Deterministic replay	<code>random_seed</code> + sorted m_i state makes $p_i = \text{score_i} / \Sigma \text{score}$ replayable; state persisted in <code>adb_sampling_frame.m_i</code> + <code>adb_collection_batches.m_state_hash</code>

All parameters (, caps, window) in `adb_collection_meta` (adaptive_version, adaptive_hash).

8.3 Zero-yield state machine (NEW, binding — V3.9-f.7, §54)

Resolves PART 1 contradiction where older wording swapped which state affects `m_i`:

State	Entry condition	Leaves frame?	Affects <code>m_i</code> ?	Next state
<code>normal</code>	Last observation had 1 flight row	No	No (normal <code>m_i</code> path)	<code>zero_yield_once</code> on first empty
<code>zero_yield_once</code>	2 consecutive empty observation	No	NO — excluded from adaptive evidence (single empty could be outage)	<code>zero_yield_repeated</code> on 2nd consecutive empty; normal on success
<code>zero_yield_repeated</code>	3 consecutive empties	No	YES, candidate for down-weight (<code>m_i</code> × 0.75 soft penalty)	<code>zero_yield_persistent</code> after 5 consecutive empties or 30 days of empties
<code>zero_yield_persistent</code>	4 persistent empties (see above)	No	Excluded from adaptive evidence (like <code>once</code>) — log for manual review, still selectable via coverage floor	Manual review may move to <code>coverage_failed</code> if feed error proven
<code>coverage_failed</code>	Feed/probe error (not empty)	YES — may leave frame	N/A	—
<code>stale</code>	No direct observation >60 days	No	No	—

One empty observation never removes airport; `coverage_failed` is the only exit.

8.4 Anchor / HUB relationship (NEW, binding — V3.9-f.7, §62)

When `ADB_ANCHOR_ENABLED=true`, the single rotating anchor **consumes the one HUB slot** in `{HUB:1, MID:2, REGIONAL:1}`. That is: each batch has 1 anchor-HUB (from 5-pool, `no-repeat-until-all`, `anchor_pool_seed`), 2 MID (freshest-first per §8.6), 1 REGIONAL (yield-aware draw §8.2). Code and docs must match; `pickAirportCandidates` enforces exactly 1 HUB when anchor enabled, otherwise draws HUB via freshest-first.

8.5 HUB and MID selection mechanics (NEW, binding — V3.9-f.7, §63)

Tier	Selection	Freshness rule	Eligibility filter	Tie-break
HUB (anchor)	<code>anchor_pool</code> rotation per §8.4	<code>no-repeat-until-all</code> 5-cycle, seeded <code>anchor_pool_seed</code>	pre_eligible && <code>post_eligible</code> && <code>in_frame</code>	Lexical ICAO if tie
HUB (if anchor disabled)	Freshest-first	ORDER BY <code>last_direct_observation_at</code> ASC NULLS FIRST, <code>tier_prior</code> DESC	Same	Same
MID	Freshest-first (deterministic)	ORDER BY <code>last_direct_observation_at</code> ASC NULLS FIRST with <code>recent_exclusion=7</code> days (no MID re-picked within 7 days unless frame < 2× slots)	Same	Same

Eligibility: `in_frame=true` + required feeds (§4.4) + not in active batch + cooldown. No unspecified randomness for MID.

8.6 Coverage floor (no-starvation) (NEW, binding — V3.9-f.7, §55)

Mathematically: **no airport with $m_i=0.25$ may have its selection probability driven to zero by adaptation**. Enforcement: $m_i \geq 0.25$ guarantees $p_i = (\text{traffic_prior} \cdot 0.25) / \Sigma \text{score} > 0$. Additional forced-eligibility: any REGIONAL airport not observed in **20 days** gets `coverage_floor_boost =`

1.5× for one draw (deterministic, seeded) to prevent starvation. After boost, `m_i` returns to computed value. Interaction with `m_i`: floor boost caps at 1.5 and does not overwrite `m_i` memory. Not a representation claim.

No-starvation clarification (A31 §58): The guarantee is mathematical (`p_i > 0`), not operational (the airport WILL be selected). In practice, an airport with `m_i = 0.25` and low `traffic_prior` may have `p_i = 0.001`, meaning it could go weeks without selection. The coverage floor boost (1.5× for 20-day unseen airports) provides operational starvation prevention. However, the boost is temporary (one draw) and does not guarantee sustained selection. If sustained selection is required, increase the boost multiplier or duration. Measure actual selection frequency at Gate 0.5 and adjust if needed.

`m_i` determinism (A31 §56): `m_i` MUST be deterministic given the same seed, history, and configuration. This means: (a) the EMA update rule (`ema_yield_i = new_yield + (1 - ) · ema_yield_i`) uses a fixed (not data-dependent), (b) the clamp `clamp(raw_m, 0.25, 1.5)` is deterministic, (c) the coverage floor boost is seeded and deterministic, (d) the zero-yield state machine transitions are deterministic. Verify determinism at Gate 0.5 by running the same configuration twice and comparing `m_i` values. Record `m_i_deterministic = true/false` in manifest.

Phase-6 adaptive initial state (A31 §59): Before Phase 6 begins, the initial state of the adaptive mechanism MUST be frozen: 1. `m_i` initial values for all airports: either 1.0 (uniform) or computed from probe data (Stage-1/Stage-2 results). Record `m_i_initial_source = 'uniform' / 'probe_data'` in manifest. 2. `ema_yield_i` initial values: either NULL (no history) or computed from probe data. Record `ema_initial_source` in manifest. 3. `zero_yield_state` initial values: all airports start in `normal` state unless probe data shows otherwise. Record `zero_yield_initial_state` in manifest. 4. `coverage_floor_boost` initial state: no boost applied at Phase-6 start (boosts are triggered by observation history). Record `coverage_floor_boost_initial = false` in manifest. 5. The initial state is FROZEN before Phase 6 and does not adapt during the run — only the EMA updates adapt. Do NOT re-initialize `m_i` during the run.

8.7 Crossover specification (NEW, binding — V3.9-f.7, §64)

Item	Frozen value
<code>crossover_group_id</code>	<code>crossover_block_id</code> per 14-day block (e.g. <code>block_001</code>)
Experimental unit	Airport-day (one airport in one window) — not flight

Item	Frozen value
Matching variables	Same <code>candidate_pool</code> (same ~12 airports), same <code>time_class</code> (UTC slot $\pm 1h$), same <code>weekday_class</code> (weekday/weekend)
Treatment	<code>window_shape</code> = 1×4h vs 2×2h vs up-to-6h (see §8 calendar)
Randomization	Seeded <code>crossover_seed</code> , randomize <code>window_shape</code> within block after freeze

Crossover randomization unit (A31 §54): The unit of randomization is **airport-day** (one airport in one window). Within a crossover block, each airport-day gets an independent `window_shape` assignment from the seeded randomization. The randomization is stratified by `time_class` and `weekday_class` to ensure balance. Do NOT randomize at the block level (all airports in a block get the same shape) — this would confound airport effects with treatment effects. Record `randomization_unit` = 'airport-day' in manifest. | Template freeze | Freeze `candidate_pool` + UTC slot + day/block + `crossover_block` BEFORE treatment randomization; treatment never depends on post-freeze observation | | Incomplete pair handling | If period-1 fails (`delivery_failure`/`budget_reached`), period-2 labeled **unmatched** and excluded from primary crossover analysis | | Order/carryover | Balanced order within block; 24h washout between crossover days |

Washout definition (A31 §55): The 24h washout means: no airport-day may be assigned a treatment window that overlaps with the previous treatment window for the same airport. For example, if airport X gets 1×4h on Monday (window 08:00-12:00), the next treatment for airport X must start no earlier than Tuesday 08:00 (24h later). The washout is measured from the END of the previous window to the START of the next window. Do NOT measure from the START of the previous window (which would be a 20h gap for 4h windows). Record `washout_start` and `washout_end` on each batch for audit. | Tagging | `crossover_group`, `crossover_period` (1/2), `window_shape`, `actual_window_hours`, `stop_reason` on `adb_collection_batches` |

Crossover feasibility (A31 §52): The HARD constraints (daily credit 1,900, one batch/day, valid window shape, crossover integrity) may be mathematically unsatisfiable for some combinations. For example: if 12 airports $\times$ 3 horizons $\times$ 2u = 72 units per batch, and the daily cap is 1,900, then ~26 batches fit — but the crossover requires exactly 2 periods per block (period-1 + period-2), and the budget must cover BOTH periods. If period-1 consumes 950 credits, period-2 must fit within 950 credits (not 1,900). The scheduler must REFUSE if the remaining budget cannot cover the required period-2. Add to §8.1 REFUSE contract: “REFUSE if remaining daily budget < expected period-2 cost.” Measure

actual costs at Gate 0.5 and re-verify feasibility.

8.8 Scheduler tie-break (NEW, binding — V3.9-f.7, §65)

Run-level constrained randomized allocation (see §8 UTC schedule): when multiple seeded candidate schedules have equal $\Sigma(n_c - \bar{n})^2$ weekday $\times$ UTC imbalance, deterministic tie-break: **lexically smallest time_window_schedule_seed wins**; no optional “may also include region/local_time” criteria unless frozen as binding soft objective. Scheduler failure mode: if no schedule satisfies HARD constraints, refuse to schedule (no window).

9. Two-stage anchor probe (budget-capped)

1. **Stage 1: Exact 12** shortlisted candidates across regions (list frozen in manifest at probe start — see §9.1), **2 h** standardized probes at matched **time_class** (UTC slot $\pm 1h$) / **weekday_class** (weekday/weekend) (never crossing in real time); record unique-flights/credit, chain-links/credit, stability. **WSSS (~331 rows/h) and OMAA (~127 rows/h) re-probed the same way as yield-reference normalization** (see §9.2) — not “calibration” for model probabilities.
2. **Stage 2: Top exactly 5** candidates by **anchor_score** (see §8) get **longer 4 h confirmation** probe.
3. Final anchor pool of **exactly 5**; station/API capacity applied as a **feasibility gate rows_per_hour 60** before scoring (never a score component). Total probe spend hard-capped within the 1,900/day budget (probe daily cap 500).

Stage-1/Stage-2 rules (A31 §41-42): - **Stage-1 duration:** Exactly 2.0 hours (watchdog 60s tick). Do NOT call a cap-truncated probe “2.0h exact” — if the probe hits the 500-credit cap before 2h, the duration is CENSORED, not 2.0h. Record actual duration as **probe_duration_minutes** and flag **duration_censored=true** if cap hit. - **Stage-2 promotion:** Top 5 by **anchor_score** from Stage 1. If fewer than 5 candidates pass the capacity gate (**rows_per_hour 60**), promote all that pass. Do NOT fabricate candidates to reach 5. - **Stage-2 duration:** Exactly 4.0 hours. Same cap-truncation rule applies. - **Budget:** Total probe spend hard-capped at 500 credits/day. WSSS at ~ 331 items/h $\times$ 2h = 662 credits EXCEEDS the cap (A31 §40). Resolve BEFORE live probing: (a) increase cap, (b) shorten duration, (c) use stop-at-cap protocol and treat duration as censored, or (d) choose a different reference airport. - **Scheduling:** Sequential, never crossing real time. Each probe must complete before the next starts. Record **probe_start_utc**, **probe_end_utc**, **probe_duration_minutes** for each probe.

9.1 Anchor probe exactness (NEW, binding — V3.9-f.7, §61)

Parameter	Frozen value
Shortlist count	12 candidates — list in manifest <code>anchor_shortlist_v1</code> with exogenous values, source, retrieval date, hash (see §4.1/4.2/4.5 for traffic/region/carrier sources)
Stage 1 duration	2.0 hours exactly (watchdog 60s tick)
Stage 2 promotion count	Exactly 5 (top 5 by Stage-1 <code>anchor_score</code>)
Stage 2 duration	4.0 hours exactly
Capacity gate	<code>rows_per_hour</code> 60 — frozen in PART 1 and <code>anchor_probe.ts:CAPACITY_GATE</code> , also in manifest
Time-class	UTC slot ± 1 hour (e.g. slot 08:00 allows probes 07:00-09:00)
Weekday-class	<code>weekday</code> (Mon-Fri) vs <code>weekend</code> (Sat-Sun) matched
Scheduling	<code>matched</code> <code>time-class/weekday-class</code> , sequential, never crossing real time; <code>anchor_probe.ts:runSingleProbe</code> enforces

9.2 Yield-reference normalization (NEW, binding — V3.9-f.7, §58-59, renamed)

The old word “**calibration**” for anchor yield is renamed “**yield-reference normalization**” to distinguish from probabilistic model calibration (ECE/Brier in §13.1).

- **Primary reference:** WSSS (Singapore Changi) — `reference_yield_wsss` measured with identical 2h protocol.
- **Fallback reference:** OMAA (Abu Dhabi) — used only if WSSS `capacity_gate` fails or WSSS probe invalid; `reference_yield_omaa`.
- **Diagnostic:** Both probed even when one is primary; report `wsss_vs_omaa_ratio`.
- **Invalid-reference behavior:** If both references fail capacity gate → probe aborts, manifest records `reference_invalid`, no scoring.

Yield standardization formula (binding):

```
component_std = clamp(candidate_component / reference_component, 0, 1)
where candidate_component {uf_per_credit, chain_per_credit, stability}
      reference_component = reference_yield_wsss.component (or omaa if fallback)
```

`clamp(x,0,1) = min(1, max(0, x))`

Clipping justification: values above reference saturate at 1.0 — intentional, because anchor yield is a **minority 20% component** of `anchor_score`; saturation prevents a single lucky probe from dominating (see §8 anchor score).

Anchor score (binding, frozen):

`anchor_score = 0.40·traffic_score + 0.20·geo_score + 0.20·carrier_score + 0.20·yield_score`
`yield_score = (uf_std + chain_std + stability_std) / 3`
`traffic_score = per §4.5 exogenous traffic normalized`
`geo_score, carrier_score per §4.5`

Station/API capacity is a **feasibility gate** before scoring, never a score component.

Stability calculation (binding):

`buckets = 15-minute buckets of 2h window (8 buckets)`
`count_i = row count in bucket i (whatever implementation counts - rows/events)`
`mean = Σcount_i / 8`
`variance = Σ(count_i - mean)2 / 8 (population variance across buckets; NOT sample n-1)`
`SD = sqrt(variance)`
`CV = SD / mean (if mean=0 → CV=0, stability=0)`
`stability = 1 / (1 + CV) clamp to [0,1]`

All buckets use same definition; formula in `anchor_probe.ts:computeStability` + manifest.

10. Weather (LOCKED)

- METAR/TAF → `observation_time / issue_time` cutoff; a TAF issued at T-2 is never used for a T-24 prediction.
- **Forecast-as-known-at-cutoff (NEW, blocking):** every forecasted weather input preserves `source`, `issue_time`, `retrieval_time`, `information_available_timestamp` (when OUR layer had it, §6.1), `valid_from`, `valid_to`, and `amendment/revision` id where available. The model sees the forecast **as it existed at the cutoff** — never the meteorological truth that became known later (TAF amendments must not leak a revised forecast into a snapshot taken before the amendment was issued). Retrospective archive availability eligibility (§6.1).
- Sources (free, no AeroDataBox credit cost — retrieval/storage/archive are separate engineering constraints): aviationweather.gov METAR/TAF (operational, **30 days history** per current AviationWeather Data API docs — verify at freeze, not 15 days; cache files), NOAA GFS/NAM grids, ERA5 / NOAA LDM reanalysis for deep backfill.
- Schema: `weather_observation` + `weather_forecast` tables; `source` tag (`live_metar` | `archive_metar` | `gfs` | `era5`) so depths never mix.

- **Weather-availability gate (new):** weather feature availability is itself measured and reported; missing historical weather is never silently filled from a later/revised source. The layer is “architected” day 1, “complete” only after the archive coverage check passes.
- Enters the ladder at **Model 2**; marginal value = Model2 – Model1 on Engine A.

10.1 Weather hierarchy and joins (NEW, binding — V3.9-f.7, §68)

Item	Frozen value
Precedence	<code>live_metar</code> (aviationweather.gov operational) > <code>archive_metar</code> > <code>gfs</code> > <code>era5</code> (reanalysis) — first available at <code>available_at</code> cutoff wins; deeper source never overwrites shallower if shallower exists
Spatial join	Nearest station within 30 km of airport reference point; if none, fallback to grid (GFS/ERA5) cell containing airport
Temporal join	Snapshot at T uses most recent observation with <code>available_at</code> T and <code>valid_from</code> T <code>valid_to</code> (for forecasts)
Issue/amendment selection	For TAF, latest amendment with <code>issue_time</code> cutoff and <code>available_at</code> cutoff; never a later amendment’s truth
Missing behavior	No observation within 6h → <code>weather_missing=true</code> , <code>weather_source=NULL</code> ; not imputed
Archive/live distinction	<code>source</code> tag preserves origin; era vs retrieval time stored separately
Versioning	<code>weather_source_version</code> , <code>weather_api_date_checked</code> , <code>weather_retrieval_date</code> , <code>weather_archive_depth_verified_at_freeze</code> in manifest

Weather LDM correction (A31 §60): The Local Data Model (LDM) correction means: when using METAR/TAF data, the observation time (`observation_time`) is the time the weather was observed, NOT the time it was reported. For example, a METAR at 14:00 UTC with `observation_time` = 13:55 UTC means the weather was observed at 13:55, not 14:00. Use

`observation_time` for temporal joins, not `report_time`. This prevents using weather that was observed AFTER the prediction cutoff.

ERA5 leak prevention (A31 §60): ERA5 reanalysis data is available with a ~5-day delay. To prevent leakage: (a) only use ERA5 data with `era5_timestamp` `prediction_cutoff - 5days` (the reanalysis must have been computed before the cutoff), (b) never use ERA5 “forecast” mode data (only “interim” or “daily” products), (c) record `era5_max_timestamp` in manifest and verify it is `prediction_cutoff - 5days` at Gate 0.5. If ERA5 data is not available for the required timestamp, set `weather_source=NULL` and `weather_missing=true`.

10.2 Weather retention wording (binding — V3.9-f.7, §69)

AviationWeather.gov docs state **previous 30 days** (not 15) as operational retention — freeze `weather_source_version/docs_API_date/retrieval_date/archive_depth_verified` in manifest at freeze, do not hard-code 15 days.

Data-retention verification (A31 §39): Verify data retention for ALL data sources before Phase 6: 1. **AeroDataBox FIDS/Schedule:** retention period unknown — verify at Gate 0.5. If provider purges historical FIDS data after N days, the historical store must capture `as-known-at-T` snapshots before purge. 2. **AeroDataBox webhook notifications:** `adb_ingest_events` is append-only and permanent (no TTL). Raw payloads + SHA-256 are retained indefinitely. 3. **AeroDataBox airborne events:** `raw_airborne_events` is append-only and permanent. Cleaned points (`clean_airborne_points`) are derived and permanent. 4. **Weather (METAR/TAF):** 30-day operational retention per AviationWeather.gov docs. Historical weather must be captured during the run or from supplementary archives. 5. **Database rows:** no TTL on any experimental table. All data is permanent unless explicitly deleted. 6. Record `retention_verified_date`, `retention_source`, `retention_period_days` in manifest for each data source.

11. Credit accounting & the canary

11.1 Three quantities per batch (source of truth = balance)

<code>notification_items_received</code>	(from the webhook payload)
<code>credits_actually_consumed</code>	(Flight Alert balance delta: <code>B_before - B_after</code>)
<code>unique_flight_rows_created_or_updated</code>	(inserted / updated / duplicates, from <code>UpsertResult</code>)

- `credits_actually_consumed` (balance delta) is the **authoritative denominator** for all marginal-value and info-per-credit claims; internal row-counting is diagnostic only. (“1 row 1 credit” is fully retracted.)

11.2 The isolated credit canary (R1 + R3 — Gate 3)

1. **Exclusivity (R1):** list subs; delete/disable every non-experimental **active** subscription; verify no foreign sub **capable of billable delivery** remains (inactive/historical records can't bill → not contamination).
2. **balance_before** + timestamp (in audit chain).
3. Subscribe to one busy airport, **maxDeliveryRetries** = 0.
4. Wait; collect. Delete subscription.
5. **Settle:** sleep a documented window, read **B_after**, then **B_after_2**; require **B_after == B_after_2** (balance stable). **B_stable** = **B_after**.
6. **C_external** = **B_before** - **B_stable**.
7. **C_internal** = Σ **notification_items**(received) from the immutable ledger.
8. Composition: notifications (POST count), items/notification, max burst, stored/inserted/updated/skipped, **delivery_failures**.
9. **PASS iff** $|\mathbf{C_external} - \mathbf{C_internal}| \leq \mathbf{tol}$ (default 3) AND failures = 0 AND balance stable AND no foreign billable sub.

Tolerance justification (A31 §33): **tol=3** accounts for: (a) balance read timing jitter (balance may change between **B_stable** read and **C_internal** computation if provider processes notifications asynchronously), (b) floating-point precision in credit arithmetic, (c) race conditions in concurrent notification processing. The tolerance is NOT a budget for silent loss — any $|\mathbf{C_external} - \mathbf{C_internal}| > 3$ requires root-cause analysis before resuming. The tolerance was calibrated from the rl3 live test (balance 2901→2900, 1 credit consumed, tolerance of 3 provides 2-credit margin for timing). Re-measure at Gate 0 with actual burst patterns. 10. **SOFT_STOP** margin tuned from this canary's measured burst.

Notification balance vs final accounting (A31 §32): **C_external** = **B_before** - **B_stable** measures credits consumed by **webhook notifications only** (the Alert subscription). This is NOT the same as the final accounting for the entire run, which includes: (a) webhook notification credits (measured by canary), (b) FIDS REST API units (measured by per-category ledger in §5.4), (c) any supplementary REST calls (measured separately). The canary proves webhook accounting correctness; the per-category ledger proves REST accounting correctness. Do not conflate these two accounting systems.

11.3 Webhook reliability & the daily cap (R2 — Gate 4)

- **maxDeliveryRetries** = 0; delivery failure → **PAUSE** the run, stop the batch (**stop_reason**='delivery_failure'), **flag** affected rows/observations (**sampling_reason**='delivery_failure' or audit column), log “reconcile before resume”. Never silently resume.
- **Failure window (A31 §30):** The time between webhook delivery failure and operator detection is UNBOUNDED (no automatic retry, no SLA on operator response). During this window, flights in the failed batch are: (a)

still in `flight_population` (they existed at cutoff), (b) NOT captured in `flight_events` (no webhook notification), (c) NOT in `flight_snapshots` (no outcome). These flights are classified as `missing_notification` — they are excluded from primary analysis but tracked for coverage diagnostics. The failure window is logged as `failure_window_start` / `failure_window_end` on the batch. Do NOT assume the failure window is bounded by the next batch start.

- `SOFT_STOP = 1900 - ADB_DAILY_SOFT_STOP_MARGIN` (default 50) stops the active batch when today's ledger spend reaches it; `HARD_CAP = 1900`; overshoot $\rightarrow$ `reconciliation_status='MISMATCH'`.

Hard-cap safety mechanism (A31 §34): If `HARD_CAP = 1900` is exceeded:

1. The watchdog immediately stops the active batch (already implemented).
 2. The overshoot batch is flagged `reconciliation_status='MISMATCH'` (already implemented).
 3. **NEW:** The overshoot credits are deducted from the next day's budget (`next_day_budget = 1900 - overshoot_amount`). This prevents cumulative overshoot across days.
 4. **NEW:** If overshoot > 100 credits, the run is PAUSED and requires operator intervention before resuming.
 5. **NEW:** The overshoot is logged as a `hard_cap_overshoot` event in `adb_collection_batches` with `overshoot_amount`, `overshoot_timestamp`, and `next_day_budget_adjustment`.
 6. Do NOT silently absorb overshoot into the daily budget without logging.
- Orphan cleanup enforces exclusivity at every batch start. - Second-start protection (one auto-started window/day) stays.

12. Model ladder, features, graph

12.1 Model ladder (each rung measured, not assumed)

- 1 Naive operational persistence (last-known state: airport recent delay, route recent delay, aircraft previous-leg delay) $\leftarrow$ the gate for deployment claims
 - 0 Calendar/seasonal baseline (no features)
 - 1 Tabular XGBoost (airport + route + aircraft + schedule stats)
 - 2 Model 1 + weather (METAR/TAF at cutoff)
 - 3 Model 1/2 + cross-sectional network (static graph)
 - 4 Model 3 + temporal/rollout aggregation (GNN) $\leftarrow$ hypothesis, not default
 - 5 Model 4 + rotations/chains (same-tail legs)
 - 6 Model 5 + disruption/event signals (cancellations, MCDs, alert flags)
 - 7 Model 6 + uncertainty/calibration (conformal) $\rightarrow$ deployed product model
- Every ladder step reported TWICE: (a) Engine-A future-representative test,
 - (b) Engine-E disruption stress. **Model —1 is the gate every ML model must clear for general-deployment claims**; the gate is engine-specific — losing Engine A may still win Engine E, and both are reported.

- Three horizons (T-24/T-6/T-90) evaluated separately; separate models (M24/M6/M90) vs shared model conditioned on `horizon_hours` is an open experiment.

12.2 Feature discipline

- **Universal eligibility rule (NEW, blocking):** `information_available_timestamp prediction_cutoff` — a feature is eligible only when our system could actually have KNOWN it by the cutoff, not merely when the event says it happened (§6.1). `event_timestamp`, `provider_published_utc`, `available_at`, `received_timestamp_utc` are preserved separately per observation and never conflated. The old shorthand `feature_timestamp prediction_cutoff` is superseded: it remains necessary but is no longer sufficient.
- **Schedule/status/destination as-known-at-cutoff (NEW, blocking):** a PRE snapshot may use **only the version of the schedule that was observable by its prediction cutoff**. `scheduled_/revised_departure_utc(at_cutoff)`, `scheduled_/revised_arrival_utc(at_cutoff)`, `destination_at_cutoff` (original vs current vs actual) are each stamped with the time the value became known. The final/revised value that surfaced AFTER the cutoff is a LABEL or test-time input only — never a snapshot feature. Same discipline applies to AIRBORNE snapshots (use state observable at observation timestamp `t`). Rationale: an airline revising departure 15:00→16:00 at 13:00 must NOT leak into a T-6, 09:00-cutoff training example built later from the current FIDS value — that leakage makes an otherwise clean dataset look artificially strong.
- Tail/rotation features: `previous-leg` counts only if landed before cutoff AND the previous leg’s arrival value was **available** by cutoff (`information_available_timestamp prediction_cutoff`, §6.1). Knowledge status is stored per previous leg: `previous_leg_exists_known`, `previous_leg_landed_known_0/1`, `previous_leg_arrival_value_known_0/1`, `previous_leg_arrival_available_at`. “Landed but value not yet in our system at cutoff” is treated as UNKNOWN, exactly like a landed-but-unreceived event.
- Explicit missingness features: `tail_known`, `days_since_last_obs`, capture flags — never silently dropped.
- **As-of historical feature store (NEW, first-class, blocking):** features like recent airport/route/tail delay, congestion, utilization and operational state need a historical lookback that a 31-day collection does NOT automatically provide at T-24/T-6/T-90m. These get an explicit immutable store, `historical_feature_store`, keyed by `(airport|route|carrier×airport| tail|OD-pair|weather) × datetime`, where EVERY row carries: `feature_value`, `source`, `source_timestamp`, `information_available_timestamp`, `valid_from`, `valid_to`. A snapshot at time T builds its feature by fetching the most-recent value with `information_available_timestamp T` — never

by computing it later “from the completed dataset”, which would be retrospective leakage (§6.1). The store is populated during the run (and by the §10 archive layers for weather), not reconstructed after.

Historical store source matrix (A31 §61-63): The historical store must document the source for each feature type:

Feature type	Source	Source timestamp	Information available timestamp	Notes
Airport delay (recent 7d)	AeroDataBox	<code>fids_retrieval</code>	<code>fids_retrieval</code>	Updated during run
Route delay (recent 7d)	AeroDataBox	<code>fids_retrieval</code>	<code>fids_retrieval</code>	Updated during run
Tail utilization (recent 24h)	AeroDataBox	<code>fids_retrieval</code>	<code>fids_retrieval</code>	Requires registration mapping
Weather (METAR/TAF)	AviationWeather	<code>observation_time</code>	<code>available_at</code>	30-day retention; archive backfill
Weather (GFS/ERA5)	NOAA/ECMWF	<code>forecast_time</code>	<code>forecast_time</code> + <code>processing_delay</code>	ERA5: ~5-day delay
Congestion (airport)	AeroDataBox	<code>fids_retrieval</code>	<code>fids_retrieval</code>	Derived from departure/arrival counts
ATC delay-program	FAA (if available)	<code>program_time</code>	<code>program_time</code>	Optional; may not be available
Storm-track	NOAA (if available)	<code>track_time</code>	<code>track_time</code>	Optional; may not be available

Record `source_matrix_version` in manifest. Verify source availability at Gate 0.5. - **Warm-up / bootstrap rule (NEW, blocking):** Day 1 of collection CANNOT build T-24/T-6 historical features from data that only starts on Day 1 — the store must be populated BEFORE the first eligible prediction snapshot. Define a **pre-run bootstrap period** (weather backfill from §10 archives + provider/FIDS state history as far back as the provider retains, plus any qualifying pre-run collection) so that the earliest evaluation day has the same historical as-of maturity as later days: `text PRE-RUN BOOTSTRAP (weather archive backfill + history as deep as provider retains)`
↓ `historical_feature_store` populated; availability frozen

(history_ready_at) ↓ Day-1 evaluation begins - first prediction cutoff is AFTER history_ready_at If/where history is still unavailable at a snapshot, the feature must NOT be silently omitted; it is stamped history_incomplete and that snapshot is excluded from the **primary** deployment evaluation (a history_complete subset is reported separately, §13). - **Hard consequence for Model —1 (persistence):** airport recent delay, route recent delay, previous-leg delay are its core inputs. Without the bootstrap, the first portion of the dataset is structurally different from the rest and Model —1 is not even well-defined. The bootstrap is therefore a pre-implementation requirement, not a nicety. - **No 1/p auto-weight.**

12.2.1 Historical feature store detailed spec (NEW, binding — V3.9-f.7, §70)

Item	Frozen value
Store name	historical_feature_store (immutable, append-only)
Key	(entity_type, entity_id, feature_name, valid_from) — entity types: airport, route, carrier×airport, tail, OD-pair, weather
Columns per row	feature_value, source, source_timestamp, information_available_timestamp, valid_from, valid_to, source_hash
Feature list	airport_delay_1h/6h/24h, route_delay_1h/6h/24h, carrier×airport_delay, tail_previous_leg_delay, OD_delay, utilization_7d, congestion_1h, weather_snapshot_id — full list in manifest
Lookback	airport/route: 7 days min, tail: previous leg within 24h, weather: 6h; window frozen
Minimum observations	airport delay: 5 flights in lookback else history_incomplete; tail: 1 previous leg landed
Update frequency	Real-time on each flight_events arrival + daily batch recompute

Item	Frozen value
Availability rule	Snapshot at T fetches <code>max(valid_from) WHERE</code>
Bootstrap source	<code>available_at</code> T — never future Weather archive backfill (§10) + provider FIDS history as far as retained (7 days) + pre-run collection if available
Readiness criterion	Operational (A30_3 #10): <code>history_ready_at</code> = earliest timestamp such that full required lookback (airport 7d, tail 24h, weather 6h per §12.2.1) is available as-of that timestamp (i.e. <code>available_at</code> <code>history_ready_at</code> for all required history) — not just <code>max(bootstrap_end,</code> <code>cutoff-lookback)</code> formula; earliest evaluation cutoff must be <code>history_ready_at</code> ; reported <code>history_ready_at</code> + <code>history_bootstrap_report</code>
Missing behavior	<code>history_incomplete=true</code> → excluded from primary (see warm-up rule above); secondary <code>history_complete</code> subset reported

12.2.2 Chain completeness (NEW, binding — V3.9-f.7, §72 — expanded A31 §6.4/§64)

`chain_completeness` = `linked_legs` / `should_have_successor_legs`. The denominator must distinguish:

Category	Definition	In denominator?
<code>successor_scheduled</code>	Tail has another scheduled departure from same airport within <code>max_turnaround=6h</code> on same service date, next leg's <code>origin</code> = previous destination	YES (denominator)

Category	Definition	In denominator?
<code>successor_observable</code>	Scheduled successor falls within the observable collection window	YES (denominator)
<code>successor_observed</code>	Successor was actually observed/linked	Numerator
<code>successor_linked</code>	Successor matched to a physical flight instance	Numerator
<code>known_absent</code>	Observed no edge (tail ended operations, cancelled successor)	Excluded from denominator
<code>unknown</code>	Unobservable (future leg outside collection window)	Excluded from denominator
<code>boundary</code>	Last leg of day / observation window boundary	Excluded from denominator
<code>aircraft_swap</code>	Registration changes mid-chain	Breaks chain (excluded)

Denominator excludes `missing_registration`, canceled next legs, `boundary`, and `unknown` (unobservable future). Report both if scientifically useful: `scheduled_chain_completeness` and `observable_chain_completeness`. Calculated weekly per `tail` and reported in §17.2 dashboard. Do not inflate failure rate by counting unobservable future legs as chain-capture failures.

12.3 Graph edge taxonomy

- **Static** (route/schedule, from catalog + schedule ref data)
- **Dynamic** (state-at-T congestion, from snapshot columns at cutoff)
- **Resource** (capacity/ops: runway/stand/ATC config, from reference + flags)
- **Aircraft/flight-chain** (same-tail legs — first-class edge type for delay propagation)
- **Missing edge** **zero edge:** `known-absent` (observed, no edge) vs `unknown` (unobserved) are distinct masks; a GNN that treats “not observed” as “no edge” learns the collection pattern, not aviation structure.

Graph edge → collection requirements (each edge must be constructible from preserved fields): - *Static route:* `flight_population` (dep/arr ICAO pairs from FIDS/schedule). - *Dynamic congestion:* `flight_snapshots` + `flight_events` state-at-T (airport delay aggregates at cutoff). - *Resource:* reference/event flags (runway/stand/ATC config preserved on the event row). - *Aircraft/flight-chain:* `aircraft_reg`-joined same-tail leg sequence from

flight_events — explicit gate-out → wheels-off → wheels-on → gate-in chain per tail, as an event history (SDSU chained-prediction support, §19).

13. Evaluation suite (Engines A–E + R + P) + Model POST

Two model families, evaluated separately (each with the full metric block):

- **PRE family:** Engines A–E + R + P (below) — the pre-departure evaluation.
- **AIRBORNE (Model POST):** trained on `flight_airborne_snapshots`, targets POST-A (landing ETA / remaining time) and POST-B (final wheels-on delay / gate-in delay). Validation preserves the time-ordered airborne ordering: observations are time-ordered per flight, and `reportedAtUtc` cutoff for every feature. Evaluation blocked by flight/date and by disruption event; never mixes with PRE rows.

Engine	Question	Blocking
A	Future/deployment-representative under the collection regime (primary; model selection + marginal-value instruments)	chronological, day/event-blocked; tails REUSABLE across train/test; disruption days at their observed frequency
B	Unseen airport (same region)	airport-level blocking
C	Unseen region	region-level blocking
D	Unseen tail / aircraft type (cold-start)	HARD tail-blocking (the only engine that does)
E	Disruption stress	whole disruption event in one partition
R (NEW)	Unseen route/OD pair — separates airport-identity memorization from general dynamics (e.g. train LAX→ORD/SEA + SFO→ORD, test SEA→JFK)	route-level holdout

Engine	Question	Blocking
P	Population audit	from FIDS census, vs Engine-A deltas
POST (NEW)	Airborne ETA / arrival-delay (Model POST, §6.2)	time-ordered per flight; flight/date + disruption-event blocked; time-anchored cutoffs

POST partition rule (strengthened): for the PRIMARY POST generalization test, **all airborne snapshots from the same flight instance stay in ONE evaluation partition** (no $t1 \rightarrow \text{train}$ / $t2 \rightarrow \text{test}$ within one flight). A SEPARATE, explicitly-labeled experiment may test future flights of known aircraft/tails (tail-held-out POST), never mixed into the primary number.

- Eval builder takes explicit `group_by` per engine (`tail` / `event_id` / `calendar_day` / `airport` / `region` / `route`); Engine A groups by `calendar_day` + `event_id`, NOT `tail`; refuses splits that break a relevant group.
- **CI via block bootstrap** at the experimental unit (calendar day / disruption event), not blanket clustered SE — frozen per §13.4: 95% CI, 1000 replicates, block unit `calendar_day` for Engine A, `disruption_event_id` for Engine E, `airport` for Engine B, `region` for Engine C, `tail` for Engine D, `route` for Engine R.
- **Engine-A test protection (CORRECTED — V3.9-f.7, §73): BEFORE Phase 6**, freeze + hash the **split-assignment rule** (calendar boundaries, blocking variables, group keys, event definition, seed, chronological ordering) — `split_rule_version`, `split_rule_hash` in manifest. **AFTER Phase 6 collection but BEFORE any model tuning**, apply that frozen rule to the actual collected rows, materialize the concrete Engine-A test row IDs, hash them (`test_row_hash`), make them read-only. **Tuning (features, hyperparameters, policy) may only use train/validation; final test never seen before.** Old wording “materialize test rows before Phase 6” is superseded — rows don’t exist yet.
- Rolling-origin on the 31 days labeled “**early rolling-origin pilot evaluation**”, never “robust seasonal validation” — folds frozen per §13.4.
- Constructible-at-cutoff unit test in the snapshot builder (a future-dated feature must error).

Rolling-origin vs final test (A31 §73): Define the distinction clearly: 1. **Rolling-origin evaluation:** Used for model development and validation during the run. Origin rolls forward daily (e.g., Day 1-15 train, Day 16 validate; Day 2-16 train, Day 17 validate; etc.). Multiple train/validation splits. Used for: feature selection, hyperparameter tuning, model comparison, early stopping. NOT used for final deployment claims. 2. **Final test:** Single

held-out test set (last N days of the 31-day run, or a pre-specified disruption event). NEVER seen during model development. Used for: final deployment claim, marginal-value instruments, collection-regime robustness test. Read once, at the end. 3. **Separation:** Train/validation (rolling-origin) and test (final) are separated by a `test_holdout_gap` (e.g., 3 days) to prevent temporal leakage. Record `test_holdout_gap_days` in manifest. 4. **Fold definition:** Rolling-origin folds are frozen per §13.4 (calendar-day blocking). Record `rolling_origin_fold_count`, `rolling_origin_fold_definition` in manifest. - **Collection-mechanism ablation (a major result, not a side diagnostic):** A = all features; B = minus coverage-age / notification counts / capture flags / observation density / sampling strategy / subscription metadata; C = minus airport identity; D = minus graph connectivity. Answers: **does the model learn aviation operations, or how we bought the data?** - **Staleness curve (restored, first-class):** prediction error vs `state_age` = `prediction_cutoff` - `last_observation_timestamp` at explicit buckets (10 min, 30 min, 1 h, 3 h, 6 h, 12 h, 24 h, 48 h, ...), for PRE and POST, on Engine A + POST. Directly answers “should I spend another credit refreshing this airport?” and grounds the coverage-age feature rather than assuming staleness is harmless. - **Collection-regime robustness test (restored):** every event retains `sampling_strategy`, `sampling_reason`, `window_shape`, seed, probabilities, `coverage_age`; the eval explicitly tests **train-on-4h → test-on-2×2h**, **train-standard → test-event-regime** (regime as a blocking/ablation factor). Separates “model understands aviation” from “model understands this particular collection regime” (§8 calendar).

13.1 First-class calibration metrics (new)

Report for every model (PRE and AIRBORNE), every horizon, on Engine A and POST (and E): MAE, RMSE, **calibration error (reliability-diagram ECE)**, **Brier score** for $P(\text{delay} > 15)$ and $P(\text{delay} > 60)$, **prediction-interval coverage**, **interval width**, **tail performance** (delay 60/120 min). The product outputs `expected_delay`, $P(\text{delay} > 15)$, $P(\text{delay} > 60)$, and a **quantile prediction interval** (Month 1) or **conformal interval** (Model 7+). For Model POST add **ETA error vs remaining-flight-time curve** (precision should improve as the aircraft approaches landing).

Metrics frozen: ECE bins=15 equal-width, Brier targets $P(\text{delay} > 15)/P(\text{delay} > 60)$, interval coverage 90% (simple quantile baseline for Month 1 if conformal deferred), tail thresholds 60/120 min — see §13.5.

13.2 Primary scientific claim (NEW, binding — V3.9-f.7, §74)

Before Phase 6 freeze in manifest:

Item	Frozen value
Primary model comparison	Model 1 (XGBoost tabular + schedule/route/aircraft stats) beats Model -1 (persistence) on Engine A
Primary target	CANDIDATE: wheels_off delay (actual_wheels_off - scheduled_wheels_off) — <i>most reliable per expectation, but if Gate 0.5 finds wheels_off/scheduled_wheels_off unverifiable, FALLBACK gate_out delay (actual_gate_out - scheduled_gate_out)</i> — predeclared before looking at outcomes (A30_3 #10)
Primary horizon	T-6h (if multiple horizons co-primary, report all; headline is T-6)
Primary Engine	Engine A (chronological future-deployment representative)
Primary metric	MAE (mean absolute error, minutes), lower is better
Practical effect threshold	2 minutes MAE improvement over Model -1 (not just statistical significance)
Uncertainty rule	95% block-bootstrap CI (see §13.4) must exclude 0
Decision	Model 1 beats Model -1 iff MAE_Model1 < MAE_Model-1 - 2 min AND CI excludes 0 on Engine A T-6h wheels_off

13.3 Endpoint hierarchy (NEW, binding — V3.9-f.7, §75)

Every endpoint labeled PRIMARY / SECONDARY / EXPLORATORY before inspecting outcomes:

Endpoint	Horizon	Engine	Label
selected_primary_target delay MAE (= wheels_off delay MAE if verified, else gate_out delay MAE fallback — frozen at Gate 0.5)	—	A	PRIMARY
gate_out delay MAE, wheels_on delay MAE, gate_in delay MAE	T-24/T-6/T-90	A	SECONDARY
P(delay>15) Brier, P(delay>60) Brier, P(delay>120) Brier	all	A/E	SECONDARY
cancellation AUROC, diversion AUROC	T-6/T-24	A/E	SECONDARY
ETA MAE, wheels_on delay MAE (POST-B1), gate_in delay MAE (POST-B2)	POST	POST	SECONDARY (POST family)
Engine B/C/D/R/P generalization gaps	—	B/C/D/R/P	SECONDARY
calibration ECE, coverage	all	A/POST	EXPLORATORY for Month 1 (unless frozen)
Individual feature ablations beyond primary	—	—	EXPLORATORY

Prevents cherry-picking headline after results.

13.4 Model-selection and chronology (NEW, binding — V3.9-f.7, §76 + §73)

TRAIN (days 1-20 deterministic) → VALIDATION (days 21-25) → MODEL/HYPERPARAM SELECTION
→ UNTOUCHED TEST (days 26-31, Engine-A test rows hashed) → ONE deployment claim read

- Hyperparameters (XGBoost `max_depth`, `learning_rate`, `n_estimators`, etc.) tuned only on VALIDATION.
- `n-fold` inside train for stability, but final selection uses validation MAE.
- Final test never influences: feature choice, missingness treatment, hyperparameters, collection policy, endpoint, threshold, model family.
- Rolling-origin folds: 5 folds expanding train window, frozen `rolling_folds` = [15,18,21,24,27] start days.
- Every run logs `split_rule_version`, `group_by`, `seed`, fold boundaries.

13.5 Probabilistic calibration and conformal (NEW, binding — V3.9-f.7, §77)

Resolves §13.1 vs §20 contradiction:

- **Month 1:** Conformal interval is **DEFERRED** to Model 7 / later. Month 1 reports simple **quantile prediction intervals** (e.g. 90% coverage via **quantile regression** on validation) with **coverage** and **interval_width** — not conformal. ECE and Brier still reported (ECE 15 equal-width bins, Brier for $P>15/P>60$).
- **Later (Model 7+):** Full conformal calibration with frozen `conformal_method` (e.g. `split-conformal`) and `target_coverage=90%`. Deferral documented in `manifest conformal_status=deferred_to_model7`.

Prevents Month 1 claiming conformal without method.

13.6 Secondary analysis freeze status (NEW, binding — V3.9-f.7, §78)

Items formally **D=DEFERRED** (frozen before analysis, not before collection) — must be frozen before looking at validation/test outcomes, with deadline before any test read:

Frozen vs deferred analysis settings (A31 §74): Two categories:
1. **FROZEN before collection (manifest):** Settings that affect data collection and cannot be changed after Phase 6 starts. These are in `adb_collection_meta` and locked at `FREEZE`. Examples: `fids_protocol`, `t_milestone`, `split_rule_hash`, `anchor_pool`, `traffic_thresholds`.
2. **DEFERRED before analysis (pre-registered):** Settings that affect model evaluation but NOT data collection. These are frozen before looking at validation/test outcomes but after collection. Examples: `conformal_method`, `holdout_fractions`, `ECE_bins`, `rolling_origin_folds`. Record in `manifest` as `analysis_setting_name = value`, `frozen_at = timestamp`.

The distinction is critical: FROZEN settings are auditable at FREEZE; DEFERRED settings are auditable before any test read. Do NOT conflate the two categories. Record `analysis_setting_category = 'frozen' / 'deferred'` on each setting.

Item	Freeze deadline	Current
Engine-B holdout % (airport)	Before validation tuning	Proposal 15% airports, stratified by region
Engine-C region holdout	Same	Proposal hold-out South America as test region
Engine-D tail holdout	Same	20% tails
Engine-R route holdout	Same	15% directed OD pairs
Block-bootstrap replicates/confidence	Before CI compute	1000 replicates, 95%
Rolling-origin folds	Before eval	5 folds
ECE bins	Before calibration compute	15 bins
Staleness buckets	Before staleness curve	10m/30m/1h/3h/6h/12h/24h/48h
Learning-curve fit	Before curve fit	$a \cdot n + c$
Missing-feature strategy per model	Before model fit	<code>missing_indicator + median</code> for XGB

14. Marginal value per credit (the final objective)

Two quantities, never conflated: - **Feature contribution:** $\Delta M_{\text{feature}} = M(\text{full}) - M(\text{without feature})$ on held-out tests (ablation — modeling result, NOT a budget result). - **Collection marginal value:** $MV_{\text{data}} = \Delta M / \Delta \text{credits}$ from **real collection interventions** (+1 obs-day WSSS vs +1 MID vs +1 REGIONAL vs +1 tail-chain vs +1 week on an existing hub), $\Delta \text{credits}$ = actual cost. Reported as “**estimated collection marginal value under randomized/paired intervention**” — never universal causal value. Repeated/paired interventions reveal diminishing returns ($MV > MV > MV \dots$).

Marginal-value claims clarification (A31 §77): The marginal-value claims must be precise: 1. **MV is per-credit, not per-flight:** $MV = \Delta M / \Delta \text{credits}$, NOT $\Delta M / \Delta \text{flights}$. Credits and flights are not 1:1 (different airports have different costs). 2. **MV is relative, not absolute:** MV measures improvement from adding data, not the absolute value of the model. A high MV means data is scarce; a low MV means data is abundant. 3. **MV is context-dependent:** MV varies by airport tier, region, time-of-day, and disruption frequency. Report MV separately for each context. Do NOT claim a single universal MV.

4. **MV is paired, not observational:** MV must be measured from randomized/paired interventions (crossover §8.7), NOT from observational comparisons. Observational comparisons confound data quality with data quantity. 5. **MV is bounded:** MV cannot be negative (adding data never hurts) and cannot exceed the improvement from perfect data (which is bounded by the irreducible error). Report MV with confidence intervals (§13.4).

Learning curves at cumulative **2k / 5k / 10k / 20k / 30k / 40k / 50k / ~58k** flight-observations (no 100k — impossible under the budget); fit **metric** = **$a \cdot n + c$** **only inside the observed domain**; extrapolation labeled “predicted, unmeasured”. Curve carries rows AND cumulative credits AND unique flights.

Learning-curve N definition (A31 §78): The N in learning curves must be defined precisely: 1. **N = unique flight-observations:** One flight at one cutoff = one observation. A flight at T-24 AND T-6 counts as 2 observations (different cutoffs, different features). 2. **N = cumulative over time:** As the 31-day run progresses, N accumulates. Day 1 contributes ~1,500-2,000 observations (4 airports $\times$ ~400-500 flights/day). Day 31 contributes ~58,000 total. 3. **N = unique flights:** A flight observed at multiple cutoffs counts multiple times. A flight observed on multiple days counts multiple times. 4. **N = credits consumed:** Credits and N are not 1:1 (different airports have different costs). Record both **N_observations** and **N_credits** on the learning curve. 5. **N thresholds:** 2k/5k/10k/20k/30k/40k/50k/~58k are APPROXIMATE. The actual N depends on collection progress. Record **actual_N** at each threshold. 6. **Fit domain:** Only fit **metric** = **$a \cdot n + c$** inside the observed domain (N **actual_N**). Extrapolation beyond **actual_N** is labeled “predicted, unmeasured” with uncertainty bounds.

15. Code to-do — final list (R1–R7, S1–S5)

#	Delta	Where
R1	Subscription-set exclusivity (orphan-cleanup at batch start; canary asserts no foreign <i>billable</i> sub; run policy deletes non-experimental active subs)	controller startBatchInner , scripts/credit_canary.ts

#	Delta	Where
R2	SOFT_STOP margin (watchdog stops active batch at 1900 - margin ; MISMATCH on overshoot)	controller config + watchdog
R3	Canary: composition, settlement (B_after==B_after_2), audit chain, exclusivity	scripts/credit_canary.ts
R4	Cost-model wording (API-unit vs credit; “deducted on SEND”)	doc §3 + controller header
R5	Delivery-failure flag + reconcile-before-resume	migration 0018, watchdog
R6	Crossover template freeze (treatment independent of post-freeze info)	scheduler/ startBatchInner , §8
R7	Versioned manifest incl. real account plan/refill	adb_collection_meta , diagnostics
S1	Provider-observable prediction population (FIDS/schedule) + coverage metrics	migration 0019, new ETL, diagnostics
S2	Raw immutable envelope (payload + hash + versions + outcome)	migration 0019, webhook store
S3	Event-log-before-state invariant	ETL ordering
S4	Provenance invariant	ETL

#	Delta	Where
S5	Airborne time-series preservation: research event log keys on (flight, carrier, locReportedUtc) so every observation survives; raw_airborne_events → clean_airborne_points → flight_trajectory → flight_airborne_snapshots pipeline; prediction_state stamped on SNAPSHOT rows (PRE/AIRBORNE) only — never on raw events (§6.1/§6.2)	new ETL, flightDat- aPrePostStore_v3.ts dedup fix, migrations 0019–0020

Status: R1–R7 **IMPLEMENTED** (code exists, see §16 gates for LIVE-VERIFIED status) incl. §34–Q accounting already implemented (ledger + canary v1). S1–S5 **IMPLEMENTED** (migrations 0019–0020, flight_events/raw_airborne_events etc.) but S1 FIDS population fetcher, S3/S4 provenance wiring (available_at, payload_sha256), and historical_feature_store remain **IMPLEMENTED schema / NOT YET LIVE-VERIFIED** (see audit §22 v3.9–f.7 gaps). S1 adds a layer; S5 fixes lastUpdatedUtc dedup loss (see §6.2). **Gate 0 code:** budget-partition report (units + credits ledgers) — LIVE-VERIFIED at Gate 0.

16. The GO gates (ALL must pass — no 60k before Gate 5)

Gate	Action	Pass criterion
0 — Budget partition (NEW)	Verify actual plan + monthly API units; refill 1 unit = 1 credit; per-refill & balance caps; census spend budgeted on the REST line; units ledger + credits ledger both tracked	plan/units/1:1 refill verified against the live account before any census call
1 — Coverage	<code>npm run coverage</code>	<code>universeCount</code> , <code>catalogInUniverse</code> recorded, sane (universe catalog)
2 — Anchor probe	two-stage standardized probe, budget-capped	scores from the frozen formula; capacity as feasibility gate; pool NOT locked before measurement
3 — Credit canary	R1 + R3 live canary	<code>C_external</code> = <code>C_internal</code> after balance-stable; failures = 0; exclusive billable set; composition reported

Gate	Action	Pass criterion
0.5 — Webhook data-content (NEW)	During the canary, verify actual payload content for a sample of deliveries: raw event carries only event_phase , event_timestamp , data_stage PRE POST (never prediction_state — that is derived on snapshots, §6.1); the four availability-rule timestamps (event_timestamp / provider_published_utc / available_at / received_timestamp_utc) are preserved, distinct, and sane (§6.1); raw airborne fields (liveLocation lat/lon/alt/gs/track/vsi/reportedAtUtc) preserved per observation; multi-point trajectories reconstructable (§5)	1 sample batch shows reconstructable trajectories, no payload-field loss, and intact distinct timestamps; prediction_state appears only on derived snapshots, never on raw events; cadence metrics (obs_per_flight , gaps) recorded — measured before any decision to add REST (§6.2)

Gate 0.5 dedicated pilot (A31 §35): Gate 0.5 is NOT just a payload inspection during the canary — it is a **dedicated pilot** that must complete BEFORE the main 31-day run. The pilot protocol: 1. Subscribe to ONE airport (e.g. WSSS) for ONE 2-hour window. 2. Verify payload content (8 milestones, 4 timestamps, raw fields, trajectory reconstructability). 3. Measure cadence (**obs_per_flight**, gaps, trajectory completeness). 4. Measure censoring grace (**arrival-notification latency P95 + margin**). 5. Verify T milestone candidate (Gate 0.5 selects **scheduled_gate_out** or fallback). 6. Verify FIDS endpoint behavior (edge inclusivity, truncation, 2×2h segments). 7. Verify **withLeg=true** semantics (opposite-movement inclusion). 8. Verify **CodeshareStatus** handling (**resolved_operator** / **resolved_marketing** / **ambiguous_unknown**). 9. Record all measurements in manifest before proceeding to Gate 4/5. 10. **Only after Gate 0.5 PASS** may the main run begin. | **4 — Webhook + cap** | R2/R5 | failures = 0, retries = 0, SOFT_STOP stops at 1,850/1,900, second-start guard works |

Gate 4 live cost (A31 §36): Gate 4 verification costs: (a) 1 batch of ~100-200 notification items (SOFT_STOP test), costing ~100-200 credits; (b) 1 second-start guard test, costing ~10-20 credits; (c) 1 overshoot test (if safe to run), costing ~50-100 credits. Total Gate 4 cost: ~150-320 credits. This is within the daily cap and does not consume probe budget. Gate 4 must PASS before Gate 5 (population/census validation) because Gate 5 requires FIDS REST calls that consume API units from the same pool. | **5 — Population/census validation** | S1 FIDS vs webhook vs (where available) reference source for a sample of airport-windows | population captured; per-stage missingness quantified and sane; Engine-A naming stays regime-qualified |

17. Step-by-step runbook (what to do, how to do it)

PHASE 0 - Code deltas

1. Implement R1 (exclusivity), R2 (SOFT_STOP margin), R3 (canary), R5 (failure flag), R6 (template freeze), R7 (manifest).
2. Implement S1-S5 (population layer, raw envelope, event-log-first, provenance invariant, airborne time-series key + snapshot pipeline) + migrations 0019-0020; fix the lastUpdatedUtc dedup loss (§6.2).
3. Implement Gate-0 budget-partition report (units + credits ledgers).
4. grep-verify: no `sampling\_weight = 1/p` stamping; `maxDeliveryRetries = 0` on all collection subs.

PHASE 1 - Gate 0

5. Log into RapidAPI; record plan, monthly units, remaining units.
6. Call GET /subscriptions/balance; record credits.
7. Make one 1-credit refill; confirm units-1 = credits+1.
8. Confirm per-refill and balance caps; confirm FIDS (2 units) spend is within the REST line budget.
9. Print the budget-partition report; commit to the manifest.

PHASE 2 - Gates 1-2

10. Run `npm run coverage`; record universeCount / catalogInUniverse.
11. Build the stratified catalog (traffic tier × macro-region + balancing variables from the fixed reference snapshot).
12. Run the two-stage anchor probe (frozen yield formula; capacity as gate); lock the 5-airport pool + scores.

PHASE 3 - Gates 3-4 (+ 0.5, the canary)

13. R1: list subscriptions; delete/disable all non-experimental ACTIVE subs; verify no foreign billable sub remains.
14. Run `scripts/credit\_canary.ts`; require balance-stable settlement (B_after == B_after_2), C_external = C_internal, failures = 0.
15. Confirm SOFT_STOP (1,900 - margin) stops the batch at ~1,850; confirm

- second-start protection; confirm delivery_failure → pause + flag.
16. ****Gate 0.5:**** inspect a sample of stored payloads - raw events carry `event\_phase` / `event\_timestamp` / `data\_stage PRE|POST` only (``prediction_state`` must appear ONLY on derived snapshots, §6.1); the four timestamps (``event_timestamp``, ``provider_published_utc``, ``available_at``, ``received_timestamp_utc``) are preserved and distinct (§6.1 availability rule); liveLocation fields preserved per observation; multi-point trajectories reconstructable (§5); record cadence metrics (obs_per_flight, median/P95/max gap). No REST decision before this measurement (§6.2).

PHASE 4 - Gate 5 (population/census validation)

17. For a sample of airport-windows, fetch FIDS/schedule; build flight_population; join with webhook events; compute per-stage missingness (population → captured → snapshots → outcomes).
18. Where available (US), spot-check against FAA/BTS-derived schedule/ops.
19. Label results "population relative to the validated AeroDataBox-supported operational frame" unless independently validated.

PHASE 5 - FREEZE (pre-Phase-6)

20. Write the versioned manifest (frame, anchor_score version, scheduler seed, anchor seed, catalog version, feature/snapshot builder SHA, real account plan/units/refill, ****plus all frozen values from §4.1/4.2/4.5/5.1-5.4/6.0/6.3-6.6/7.1-7.4**).
21. Freeze + hash the ****split-assignment rule**** (calendar boundaries, group_by, seed, chronology).

PHASE 6 - The 31-day run (§8 calendar)

22. Day 1-5: 4h templates. Day 6: 2×2h matched to day 5. Day 7-10: 4h. Day 11: 6h (up-to-6h) matched to day 10. Day 12-13: 4h. Day 14: 2×2h matched to day 13 → checkpoint 1. Day 15-20: 4h (first snapshot ETL cut + data-quality report). Day 21: 6h → checkpoint 2. Day 22-28: 4h (weekly diagnostics). Day 29: 2×2h matched to day 28 → checkpoint 3 → default decision. Day 30-31: 4h wrap-up (export, snapshot ETL rerun, baselines).
23. Daily: watchdog enforces 1,900 cap; SOFT_STOP margin; exclusive set; delivery_failure → pause/flag; weekly diagnostics (info-per-credit, coverage-age 5 d core, new-info/credit trends, hour spread).
24. Monthly: airborne cadence metrics re-measured (obs_per_flight, gaps, trajectory completeness) so the POST decision stays evidence-based (§6.2).

PHASE 7 - Month-1 deliverables (§17.1) + evaluation

25. Build flight_snapshots ETL + leakage-safe evaluation (all engines, all horizons, calibration metrics, collection-mechanism ablation).
26. Fit Model -1 (persistence) and Model 1 (XGBoost) on Engine A; report whether ML beats the persistence gate (engine-specific).
27. Report info-per-credit curves, Engine B-E + R light results, census coverage metrics, and the collection-mechanism ablation.
28. Label all month-1 outputs ****"early operational pilot"**, never "validated"**

- production model". GNN is phase 2.
29. ****Model POST pilot:**** train airborne ETA / arrival-delay model on flight_airborne_snapshots (Engine POST, §13); report ETA-error-vs-remaining-time curve + cadence report; gate any larger POST program on this pilot.
-

17.1 Month-1 deliverables

1. Validated collection pipeline (exclusive set; accounting reconciled).
 2. Validated snapshot pipeline + provider-observable population layer.
 3. Leakage-safe **XGBoost that beats the Model —1 persistence gate** on Engine A (and per-engine results).
 4. Information-per-credit curves (using C_actual, not rows).
 5. Engine B–E + R light results; **collection-mechanism ablation**.
 6. Census coverage metrics + calibration metrics (all horizons).
 7. Window-experiment **pilot** evidence gating a properly-sized Month-2 study.
 8. **Model POST pilot** (airborne ETA / arrival-delay) + cadence/trajectory-completeness report (S5), with a go/no-go for any larger POST program (no REST before measurement, §6.2).
 9. **Regime-robustness + staleness + chain-depth results** (§17.2): the collection-regime robustness test, the staleness curve, and the aircraft-chain evidence that the propagation layer is actually rich enough.
-

17.2 Full diagnostic dashboard & chain-depth metrics (re-stored)

Every weekly diagnostics run (and the month-1 report) reports ALL of:

credits consumed	unique flights / airports
new airports / new routes / new tails	per-tier and per-region shares
PRE snapshots / credit	POST (airborne) observations / credit
delay events / severe delays (60/ 120 min)	tail-missing %
route-direction coverage	coverage-age distribution (by tier/region)
tail observations	consecutive tail-pairs (same aircraft, back-to-back legs)
3-leg chains	4+-leg chains
chain completeness = chained legs / legs that had a successor in-window	

Why chains matter: a core hypothesis is that continuous 4h windows preserve aircraft-chain continuity — the dashboard must prove **how much chain continuity was actually obtained** (3-leg and 4+-leg counts, chain completeness), otherwise the SDSU-propagation modeling claim has no evidence. **tail-chain links / credit** is the cost-normalized yield metric for the window-regime comparison.

Window-regime comparison table (per window shape: 4h / 2×2h / 6h):

unique flights / credit	unique tails / credit	tail-chain links / credit
3-leg chain count	4+-leg chain count	new routes / credit
PRE snapshots / credit	POST outcomes / credit	hour uniformity
coverage-age achieved	environmental context recorded (§8)	

18. Contradiction map (this file vs V3_CollectionStrategy2.md)

Where anything in the revision-history file conflicts, the right-hand column here governs:

History-file statement	Binding resolution
§11 “snapshot only if we hold an event after cutoff”	Snapshot existence is population-defined (§5 here)
“null = censored” in any diagram	Five-state outcome model (§7 here)
“FIDS census / true census”	“provider-observable prediction population” (§5 here)
“60k → 58,900 credits” with no REST partition	Two-budget partition + Gate 0 (§3 here)
“80/10/10”	$26 \times 4h + 3 \times 2 \times 2h + 2 \times 6h = 31$ days 84/10/6 (§8 here)
“balanced at weekday×UTC”	constrained randomized allocation, $\min \Sigma(n_c - \bar{n})^2$ (§8 here)
“persistent core”	rotating anchor pool (§8 here)
“sampling_weight = 1/p”	NULL by default; no auto 1/p (§8 here)
“1 row 1 credit”	three-quantity accounting, balance delta authoritative (§11 here)
Reserve “1,100”	57,900 spendable experimental envelope / 1,000 protected floor (inside the 58,900 refill) / 1,000 REST + 100 unallocated = 60,000 (§3.2 here); “1,100” is legacy
Engine A “future-representative”	“future/deployment-representative under the collection regime ” (§13 here)
Weather “free”	“no AeroDataBox credit cost; engineering constraints separate” (§10 here)

19. Sources & research foundation

The plan’s central scientific bets are grounded in peer-reviewed aviation research and the provider’s own documentation (all verified 2026-08-12):

Idea in our plan	Source
Previous-leg / late-arriving-aircraft delay is among the most predictive features; delay propagates along the same aircraft’s itinerary	Chen, J. & Li, M., “Chained Predictions of Flight Delay Using Machine Learning,” AIAA SciTech 2019 (San Diego State University) — junchen.sdsu.edu/proceedings/scitech_gnc19_Chen.pdf
Delay propagation differs by aircraft utilization; previous delays, turnaround/buffer, weather, and utilization influence later flights; stronger propagation in later legs	Zheng, Z., Wei, W., et al., “A Comparative Analysis of Delay Propagation on Departure and Arrival Flights,” SJSU ScholarWorks #2410 / Aerospace (MDPI) 8(8):212, 2021 — works.sjsu.edu/faculty_rsca/2410
Aircraft-chain continuity as first-class modeling; focused information-rich features beat feature-stuffing — use SDSU Chen & Li 2019 + SJSU Zheng Wei Hu 2021 for chain/propagation, NOT #4774	Zheng, Zou, et al., “A Data-Light and Trajectory-Based Machine Learning Approach...,” SJSU ScholarWorks #4774 — correctly cited ONLY for AIRBORNE trajectory ETA, not for same-tail downstream propagation (see §22 correction)
GNN for delay propagation = defensible hypothesis, not guaranteed winner	Wu, Chen, et al., “Delay Prediction of Flight Operation Network Based on Deep Learning,” SJSU ScholarWorks #4935 (GCN-GRU); ERAU 2025 AIAA GNN-for-weather-delay (portfolio.erau.edu)
Persistence / autoregressive baselines as the first gate for aviation delay	Chen & Li (2019); Sternberg et al., “A Review on Flight Delay Prediction,” arXiv (2017)
Ops + capacity/demand + wind/visibility + en-route weather	“Airport Delay Prediction with Temporal Fusion Transformers,” arXiv 2405.08293 (2024)
Delay as a network/propagation problem; data & methods taxonomy	“Flight Delay Propagation Modeling: Data, Methods, and Future Opportunities,” Transportation Research Part E (2024), ScienceDirect S1366554524001169

Idea in our plan	Source
Two-budget accounting: API quota vs Flight-Alert credits; 1 credit per flight item on SEND; retries cost extra; refill 1 unit = 1 credit; per-plan caps	AeroDataBox, “Flight Alert API: Guide to the New System” (2026-01-31) — aerodatabox.com/flight-alert-api-2026; doc.aerodatabox.com/rapidapi.html#/operations/RefillBalance
Weather data availability (METAR/TAF, 30-day operational retention per current AviationWeather Data API , free — verify at freeze)	AviationWeather.gov Data API — aviationweather.gov/data/api — record <code>weather_source_version</code> , <code>docs_API_date</code> , <code>retrieval_date</code> , <code>archive_depth_verified</code> in manifest (§10.2)
In-flight ETA/remaining-time as a first-class prediction state with trajectory features (grounding for Model POST, §6.2/§13)	Springer 2024 in-flight ETA study (trajectory-derived ETA/remaining-time, mirrors our <code>raw_airborne_events</code> → <code>clean_airborne_points</code> → <code>flight_trajectory</code> → <code>snapshots</code> pipeline); SJSU airborne-ETA line (#4774)
Airborne fields (lat/lon/alt/groundSpeed/track/vsi/repositoryId/b/disruption/flightStatus_v3.ts, <code>data_stage</code> PRE POST) are provided per notification	verified in our own code: <code>standardV1/b/disruption/flightStatus_v3.ts</code> , <code>flightNotificationExtractor_v3.ts</code> (2026-08-12)

20. Explicit NOT-do list

- **No V3.10/V3.11/V3.12** theoretical reviews.
- No GNN-first; no `1/p` weighting yet (measure the denominator first).
- No claim: “6-day slot-once unbiased”; “future-representative population-representative”; “4h statistically beats $2 \times 2h$ ” from pilot data; “31 days seasonality”; “+1 MID caused 0.7 min improvement” (marginal value is estimated under intervention, not universal).
- No foreign active subscription during the run (R1).
- No raw-event overwrites (S2–S5); the dedup `flight_state` is never the trajectory source (S5, §6.2).
- No REST-airborne monitoring before cadence measurement (Gate 0.5, §6.2).
- No merging PRE and AIRBORNE rows into one modeling set; POST labels are never finalized before a terminal event arrives (§6.1/§6.2).
- No post-cutoff schedule/status/destination revision used as a snapshot feature — only the version observable at cutoff (as-known-at-cutoff, §12.2).

- No “[window ended] = [no outcome]” — censoring requires the documented grace interval (§6.2).
- No manual Rescore / Simulate / “Rescore all” during the run.
- No silent weather backfill from later/revised sources.
- No assumption that one empty observation (or one stale day) is evidence an airport is useless — triage requires once/repeated/persistent (§4).
- No claim that the crossover equalizes the environment — context is recorded, not assumed away (§8).
- Month 1 does not switch on: event-sampling regimes at scale, IPW, advanced conformal/GNNs, long-horizon seasonal eval, intervention optimization.

21. Final status

Architecture: LOCKED (GO). Sampling: GO (experimental allocation) but FRAME NOT YET REBUILT — tier/region pending (§4.1/4.2). Research/eval: GO with PREREGISTRATION INCOMPLETE — split rule / primary claim / endpoint hierarchy now frozen in §13.2-13.6 but manifest not yet written. Leakage: GO + population layer + availability-time rule (see §6.1/6.4) but available_at wiring NOT YET LIVE-VERIFIED. Credit model: GO after Gate 0 + canary (Gate 0.5 will verify). Airborne: GO to preserve + measure (S5, Gate 0.5); REST-based airborne monitoring stays a decision AFTER cadence measurement (§6.2). Strat2 carry-forward: COMPLETE (all eight restored safeguards, §22). Implementation lock: ARCHITECTURE LOCKED; CODE IMPLEMENTED for R1-R7/S1-S5 schema; **MANIFEST VALUES PENDING** — all V3.9-f.7 frozen values (traffic thresholds, region mapping, FIDS protocol, T milestone, flight_instance_id, 8-milestone mapping, label observability, censoring grace measure→freeze, adaptation m_i, coverage floor, balancing refs, anchor formulas, stability, capacity gate, weather hierarchy, historical store readiness, split rule) must be written to adb_collection_meta before Phase 6. The four blocking fixes (Gate 0.5 wording, schedule-as-known-at-cutoff, ADB_BATCH_BUDGET=1900, censoring grace interval) + the five data-contract requirements (availability-time rule, milestone-explicit targets, POST population denominator, as-of historical store, forecast-as-known-at-cutoff — all in §6.1/§6.2/§7/§10/§12.2, V3.9-f.5) + the accounting/bootstrap/foundation resolutions (**spendable envelope = 57,900 credits (1,000 reserved inside the 58,900 refill, §3.2)**, history_ready_at (§12.2), airborne_eligible pinned (§7), milestone verification (§6.2 — V3.9-f.6)) + the V3.9-f.7 pre-freeze patch (37.1-37.4 through 78, see §22) are binding. 60k: WAIT on Gate 0 + gates 1–5 + FREEZE.** This document is the FINAL, binding spec — all previous revision files (V3_CollectionStrategy.md, V3_CollectionStrategy2.md, V3_WebhookExtractionPlan.md, CGTAnalysis*.md, ChatGptAnalysis*.md)**

are history/adjudication only and are NOT needed to execute. The remaining uncertainty — supported-universe size, effective window regime, staleness, XGBoost vs persistence, airborne cadence, marginal value of a credit, and collection-vs-aviation confounds — is exactly what this run is designed to measure. Implement, verify the budget partition, canary, validate the denominator, then collect.

22. Adjudication record: airborne + Strat2 carry-forward + implementation-lock fixes (V3.9-f.2 / f.3 / f.4)

Reviewed: our ChatGPT airborne claims (that AeroDataBox provides lat/lon/alt/groundSpeed/track/vsi/reportedAtUtc and `data_stage` PRE|POST, and that we can train an in-flight ETA / arrival-delay model). **Adjudication: the field claims are TRUE and verified against our own code** (`flightStatus_v3.ts` defines `liveLocation`; `flightNotificationExtractor_v3.ts` flattens all of them to `loc_*`, `has_live_location`, `data_stage` PRE|POST). Three NEW requirements fall out:

1. **Preserve the time series, not just the latest state.** Current dedup key `SHA-256(flight|carrier|lastUpdatedUtc)` overwrites earlier points when the provider updates location under the same `lastUpdatedUtc`. → **S5** (§15): research event log keys on (`flight`, `carrier`, `locReportedUtc`); pipeline `raw_airborne_events` → `clean_airborne_points` → `flight_trajectory` → `flight_airborne_snapshots` (§6.2).
2. **Dual prediction-state contract.** PRE (T-24/T-6/T-90m, cutoff scheduled departure) and AIRBORNE (cutoff = the observation timestamp `t`, target = ETA/ remaining time/wheels-on delay/gate-in delay) are separate modeling sets; universal cutoff rule `feature/provider_timestamp prediction_cutoff` (§6.1). POST labels never finalized before a terminal event arrives (§6.2).
3. **No REST before measurement.** Cadence is measured (Gate 0.5, §16: `obs_per_flight`, gaps, completeness) before any decision to add REST-based airborne monitoring (§6.2).

Second pass (V3.9-f.3 — Strat2 carry-forward, final lock): re-read the full candidate plan vs `V3_CollectionStrategy2.md`; verdict: architecture is a sound foundation for both PRE and POST, no redesign needed. Eight Strat2 execution-spec safeguards were partially compressed on copy-forward and are now **restored**:

Restored safeguard	Where it lives now
A. Full coverage taxonomy (supported/eligible/directly_subscribed/recently_observed/edge_discovered/zero_yield_once/re + coverage-age distribution; edge-discovered directly observed)	§5
B. Zero-yield triage: one empty observation is never treated as evidence an airport is useless; only coverage-failed leaves the frame	§4
C. Staleness curve (prediction error vs state_age , 10 min → 48 h buckets)	§13
D. Collection-regime robustness: train-4h→test-2×2h, train-standard→test-event-regime, regime metadata retained per event	§13
E. Full diagnostic dashboard + chain-depth metrics (3-leg / 4+-leg / chain completeness / tail-chain links-credit) + window-regime comparison table	§17.2
F. Crossover environmental context (weather severity, ATC delay programs, storm-track) recorded in batch metadata	§8
G. Event-vs-prediction-state distinction: events carry immutable event_phase/event_timestamp/data_stage ; prediction_state is derived on snapshots/training examples	§6.1
H. POST partition rule: all airborne snapshots from one flight instance stay in ONE partition for the primary test; tail-held-out POST is a separate, labeled experiment	§13

Enforced by: §6.1/§6.2 (contract + foundation), §13 (Engine POST + staleness + regime robustness), §15 (S5), §16 (Gate 0.5), §17/§17.1/§17.2 (runbook + deliverables + dashboard), §19 (sources), §20 (NOT-do), §21 (final status). This file is the binding spec; the adjudication record lives here, not in `V3_CollectionStrategy2.md`.

Third pass (V3.9-f.4 — implementation-lock patch): a fresh full-file read verified the foundation end-to-end; verdict: **no further architecture redesign** — implementation + preflight are the next step, not the 60k. Four blocking fixes

were made before implementation is considered locked:

Fix	What changed	Where
1. Gate 0.5 contradiction	Raw webhook events carry <code>event_phase/event_timestamp/data_stage</code> only; <code>prediction_state</code> appears ONLY on derived snapshots — Gate 0.5 verifies exactly that, so devs can't misinterpret	§16 Gate 0.5, §17 step 16
2. Schedule-as-known-at-cutoff (blocking leakage)	A PRE snapshot may use only the schedule/status/destination version observable by its cutoff; values are stamped with when they became known; post-cutoff revisions are labels/test-time only (e.g. a 13:00 15:00→16:00 revision never leaks into a T-6 09:00-cutoff snapshot)	§12.2
3. ADB_BATCH_BUDGET ambiguity	Default changed from 3000 → 1900, equal to and never above the binding daily cap; controller refuses a batch budget above today's remaining spend	§3.3/§11 (binding), DD-R §28
4. Censoring grace interval	A flight is labeled <code>active_censored/missing_outcome</code> only after a defined grace period (proposal: 60 min or provider arrival-notification latency + margin, measured at Gate 0.5); never “window ended = no outcome”	§6.2

Gate order after fixes (unchanged from §17): implement S1–S5 + R1–R7 →

Gate 0 → 1 coverage → 2 anchor → 3 canary → 0.5 webhook content/airborne
→ 4 reliability+cap → 5 population → FREEZE manifest+test → 31-day run.
60k spend remains gated on Gates 0–5.

Fourth pass (V3.9-f.5 — data-contract blocking patch): a fresh full-file review (SJSU/SDSU/FAA-grounded) confirmed the PRE/POST separation, population layer, immutable raw storage, airborne trajectory preservation, aircraft-chain and evaluation design are fundamentally correct and need NO redesign. It recommended a data-contract patch before the 60k, not before implementation. Five pre-collection blocking requirements were added, all in PART 1:

Fix	What changed	Where
1. Availability-time leakage (blocking)	A fea- ture is eligible only when <code>information_available_timestamp</code> <code>prediction_cutoff</code> — surviving that the event <i>happened</i> before cutoff is NOT enough; the value must have been <i>in our system</i> by cutoff. <code>event_timestamp</code> , <code>provider_published_utc</code> , <code>available_at</code> , <code>received_timestamp_utc</code> preserved separately, never conflated	§6.1, §12.2
2a. Milestone-explicit delay targets (blocking)	FAA-ASPM 8-field milestone set (<code>scheduled/actual</code> <code>gate_out</code> , <code>wheels_off</code> , <code>wheels_on</code> , <code>gate_in</code>); departure delay = <code>gate_out</code> AND/OR <code>wheels_off</code> as separate columns; arrival = <code>wheels_on</code> AND <code>gate_in</code> separately; <code>label_arr_delay</code> deprecated as ambiguous	§6.2, §6.2 POST labels

Fix	What changed	Where
2b. PRE-side target sync	DD-A historic <code>dep_runway_utc</code> - <code>dep_scheduled_utc</code> marked [SUPERSEDED] → milestone-explicit (§6.2)	§DD-A
3. POST population denominator (blocking)	Formal funnel mirroring §5: <code>airborne_eligible</code> → <code>observed</code> → <code>usable</code> → <code>trajectory_complete</code> → <code>snapshot_eligible</code> → <code>labeled</code> , reported monthly per airport/tier; no POST training on the easy-to-track subset silently	§7
4. As-of historical feature store (blocking)	First-class immutable <code>historical_feature_store</code> (air- port/route/carrier×airport/tail/OD/weather × time) with <code>value</code> , <code>source</code> , <code>source_timestamp</code> , <code>information_available_timestamp</code> , <code>valid_from</code> , <code>valid_to</code> ; <code>snapshot(T)</code> uses most-recent value with <code>available_at T</code> , never a post-hoc computed “historical delay”	§12.2
5. Forecast-as-known-at-cutoff (blocking)	Weather inputs preserve <code>source</code> , <code>issue_time</code> , <code>retrieval_time</code> , <code>information_available_timestamp</code> , <code>valid_from</code> , <code>valid_to</code> , <code>amendment</code> <code>id</code> ; model sees the forecast as it existed, never meteorological truth revealed later	§10

Part 2 header hardened to say **NON-NORMATIVE HISTORICAL CONTEXT - DO NOT IMPLEMENT FROM HERE** (impl-safety convention; PART 1 governs always). Gate 0.5 now additionally verifies availability timestamps on raw payloads (§17 step 16). Verdict: **architecture GO — data contract patched — implementation next, 60k ONLY after Gates 0–5.**

Fifth pass (V3.9-f.6 — accounting + bootstrap + denominator pinning): a follow-up full-file review re-read the patched plan and found three methodological items and one field-validation item to resolve before implementation (no architecture change). All applied in PART 1:

Fix	What changed	Where
1. 58,900 vs 1,000 floor (blocking, HIGH)	The 1,000 reserve is INSIDE the 58,900 refill → spendable experimental envelope = 57,900 credits (run-total binding invariant); “58,900” always means refill size, “57,900” means spend; final day sizes down automatically to stay 57,900 (e.g. days 1–30 × 1,900 = 57,000 → day 31 ~900); legacy “~1,100 reserve” wording marked [SUPERSEDED]	§3.2, DD-F §13

Fix	What changed	Where
2. Historical-store warm-up/bootstrap (blocking, HIGH)	Pre-run bootstrap period (weather archive backfill + provider history) BEFORE the first eligible prediction snapshot; <code>history_ready_at</code> frozen; any snapshot with missing history is stamped <code>history_incomplete</code> and excluded from PRIMARY evaluation (a <code>history_complete</code> subset is reported separately); Model <code>-1</code> persistence baseline explicitly requires this	§12.2
3. <code>airborne_eligible</code> denominator source (MED-HIGH)	<code>airborne_eligible = flight_population</code> (FIDS, §5) <code>actual_wheels_off</code> (or <code>provider terminal state</code>) — defined independently of airborne observation capture (using the webhook would be circular with <code>airborne_observed</code>)	§7

Fix	What changed	Where
4. Milestone-mapping verification (MEDIUM)	Each of the 8 §6.2 milestones must be verified against actual AeroDataBox JSON semantics (FlightAirportMovementContract: scheduledTime/revisedTime/predictedTime/runwayTime/gate) before schema freeze; never rename a provider scheduledTime unless semantics support it; unverifiable milestones stay NULL + marked milestone_unverified; Gate 0.5 re-verifies on live payloads	§6.2

Verdict: **architecture GO — accounting/bootstrap fixed — START IMPLEMENTATION; 60k ONLY after Gates 0–5.** All other reviewed areas (avail- ability-time rule, schedule-as-known-at-cutoff, milestone-explicit labels, POST denominator funnel, forecast-as-known-at-cutoff, weather, evaluation) confirmed correct — no further redesign.

Sixth pass (V3.9-f.7 — A30 pre-freeze patch, Aug 30): Full reconciliation of A19/A30 analyses (chatGPTv3_A19_1 30 sections, chatGPTv3_A30_1 77-item #70, chatGPTv3_A30_2 FINAL MASTER 88 sections, cgtAnalysis13 50+ sections) against PART 1 + live repo audit (migrations 0017-0023, build_stratified_catalog.ts, anchor_probe.ts, adbCollectionController_v3.ts, flightDataPrePostStore_v3.ts, flightNotificationExtractor_v3.ts).

Overall assessment (ours): Architecture is fundamentally sound — PRE/POST separation, population layer, 4-timestamp availability, immutable provenance, airborne preservation, aircraft-chain, weather-as-known-at-cutoff, anti-leakage, XGBoost-first/GNN-later, Engines A-E/R/P, collection-ablation, staleness — all validated and correctly retained. **NO V3.10 needed.** The 77-item list is not 77 defects — it is 12 blocker families where PART 1 was underspecified vs code. Real blocker count = ~12 families, not 77. ChatGPT correctly flagged that 4,053 REGIONAL blanket, ICAO-first-letter region, undefined flight_instance_id/codeshare, incomplete FIDS protocol, ambiguous T, and missing adaptation math were **genuine Phase-6 blockers**; correctly identified censoring grace / cadence / completeness as **measure→freeze** (not invented), and conformal/holdout fractions as **deferrable** with preregistered deadline. We agree with ~90% of findings; the

remaining 10% is over-strict phrasing (e.g. every GNN edge formula need not block collection if raw data preserved).

All V3.9-f.7 patches applied in PART 1:

#	Blocker family	Change	PART 1 §
37.1	Traffic tier 4053 REGIONAL blanket	Replaced with measured external traffic metric (§4.1): source/period/thresholds/HUB- MID- REGIONAL/missing pol- icy/version/hash, rebuild frame	§4.1
37.2	Region ICAO heuristic	Replaced with country→macro- region lookup table, 6 regions, exceptions, unknown policy, version/hash	§4.2
37.3	Population scope cargo/private/canceled	Frozen table (commercial YES, cargo/private/charter NO, canceled/diverted YES, marketing deduped)	§4.3
37.4	PRE/POST eligibility	Frozen Option B: HUB/MID require pre_eligible && post_eligible, REGIONAL post_eligible preferred	§4.4
38	Exact T milestone	Frozen T = scheduled_gate_out UTC, provider path, original vs revised, service_date, retime	§6.0

#	Blocker family	Change	PART 1 §
39	T-24 acquisition	Forbidden reconstruction; live snapshots with 25h assignment freeze, scheduler obligation, alternative proof	§5.2
40	Complete FIDS protocol	Frozen end-point/direction/withLeg/withCancelled/withCodeshared/withCargo/window 12h, pagination, dedup, raw hash	§5.1
41	FIDS timezone DST	Frozen UTC→IANA→local DST-aware conversion, persist both, mandatory tests	§5.3
42	REST budget proof	Explicit worst-case 899 < 1000 formula: BASE 744 + VALIDATION 60 + DIAGNOSTICS 20 =824, +75 retry cap =899 (enforced)	§5.4
43	Canonical flight_instance_id	Frozen operating carrier/number/origin/destination/service_date/scheduled_gate_out + code-share/retime/diversion/collision rules, separation stable vs mutable	§7.1
44	Codeshare dedup	Marketing → operating canonicalization, store marketing as attribute, Gate 0.5 verify	§7.2

#	Blocker family	Change	PART 1 §
45-46	Route OD / Tail identity	Directed OD, original vs actual, diversion, priority reg>mode_s>icao24	§7.5
47	8 OOOI milestones	Per-milestone provider path/semantics/caveat, NULL+milestone__unverified if unverified	§6.3
48	Target-specific labels	Generic observed → gate_out/wheels_off/wheels_on/gate_in_label_observed	§7.3
49	Censoring	Full sequence with frozen grace, patches stale wording	§7.4 + §6.2
50	Four timestamps	Separate table with nullability/order/QC, universal available_at cutoff	§6.4
51	Snapshot provenance	Provenance JSON/hash linking to raw/population/weather/history/builder	§6.5
52	Gate-0.5 measure→freeze	Table of 10 items (grace, N usable, cadence, max gap, duration, completeness, warning/fail)	§6.6 + §16
53	REGIONAL m_i	Full recurrence ema =0.5, smoothing, window, cold start, reset, cap [0.25,1.5]	§8.2
54	Zero-yield FSM	5-step machine, what affects m_i, what doesn't, coverage_failed only exit	§8.3

#	Blocker family	Change	PART 1 §
55	Coverage floor	No-starvation guarantee, m_i 0.25, forced boost if 20d unseen	§8.6
56	Balancing vars	Degree/carrier/intl/ §4.5 geo/carrier scores with	§4.5
57-60	Anchor formulas	source/formula/period/missing 40/20/20/20 frozen, yield_reference normalization rename, component_std clamp justification, stability 1/(1+CV) population variance	§9.2 + §4.5
61	Probe exactness	12 shortlist, 2h/4h, top 5, capacity 60, time/weekday class	§9.1
62	Anchor=HUB slot	Explicit anchor consumes HUB slot	§8.4
63	HUB/MID mechanics	Freshest-first, 7-day exclusion, tie-break	§8.5
64	Crossover	Full spec: group, unit airport-day, matching, treatment, seed, incomplete handling	§8.7
65	Scheduler tie-break	Lexically smallest seed	§8.8
66	Coverage-age 5d core	Recomputed per batch, diagnostic subset	§4.6

#	Blocker family	Change	PART 1 §
67	Env context	Weather severity/ATC/storm-track captured per batch (already in §8)	§8
68-69	Weather hierarchy/retention	Precedence live>archive>gfs>era5, 30km/6h joins, 30-day retention	§10.1-10.2
70	Historical store	Full spec: key, columns, lookback, min obs, bootstrap, history_ready_at	§12.2.1
72	Chain completeness	Denominator with 6h turnaround, boundary, swap handling	§12.2.2
73	Engine-A chronology	BEFORE freeze rule hash, AFTER materialize row IDs before tuning	§13 + §17 PHASE 5
74	Primary claim	Model1 vs Model-1 on T-6 wheels_off MAE 2 min + CI	§13.2
75	Endpoint hierarchy	PRIMARY/SECONDARY/EXPLORATORY table	§13.3
76	Model selection	Train→validation→test, rolling folds [15,18,21,24,27]	§13.4
77	Conformal	Month1 DEFERRED to Model7, quantile 90% interim, frozen method later	§13.5
78	Secondaries D	Table with deadlines before analysis	§13.6

Repo truth vs PART 1 (audit Aug 30): traffic tier/region provisional but now specified — needs code patch + frame rebuild; FIDS `flight_population` schema exists but fetcher not implemented — **Phase 6 blocker until coded**; `historical_feature_store` absent — **blocker**; weather tables absent — **blocker**; `flight_instance_id/codeshare` not canonicalized — **blocker**; `m_i` adaptation stub — **blocker** (currently uniform); `available_at/payload_sha256` not wired — **partial**; evaluation Engines not coded — **deferred to analysis but primary Engine A split rule must be frozen now**. All correctly classified B/C/D per A30 — unresolved B count must reach 0 before Phase 6.

Verdict (ours, reconciled):

Question	Answer
PRE + AIRBORNE architecture wrong?	No — sound, SJSU/SDSU/FAA/Transportation Research Part E 2024 all support core bets, with citation corrections (§19: #4774 only for AIRBORNE, not chain)
Need V3.10 redesign?	No — patch V3.9, don't redesign
Every numeric constant proven by literature?	No, and need not be — 40/20/20/20, 60 rows/h, 1,900/day, {1,2,1}, 4h vs 2×2h are project-specific experimental choices , not literature constants — correctly frozen as predeclared design
Anchors proven optimal by SJSU/SDSU?	No — anchors are allocation mechanism, 20% yield minority guards against feedback loop; choice justified as experimental design, not theorem
Architecture GO?	YES
Frame frozen?	NOT YET — needs external traffic/region sources + rebuild (§4.1/4.2)
Population/FIDS validated?	NOT YET — fetcher missing, Gate 5 pending
Evaluation preregistered?	NOW YES (patched §13.2-13.6) — rule frozen, row IDs deferred
Phase 6 GO?	NO-GO — until manifest written, Gates 0/1/2/3/0.5/4/5 pass, frame rebuilt, FIDS/fetcher/history/weather coded, canary PASS after <code>is_randomized</code> fix

Next order (see runbook §17): code the FIDS/historical/weather/flight-id patches → rebuild frame (4.1/4.2) → Gates 0→1→2 (cover/anchor) → canary (must PASS) → Gate 0.5 measure cadence/grace → Gate 4/5 population → manifest → preflight `proposal/TBD/~ scan via scripts/v39_preflight_consistency.ts` (CURRENT_CONTRADICTIONS=0) → Phase 6.

Seventh pass (V3.9-f.8 — A30_3 10 consistency fixes, Aug 31): Full consistency audit of V3.9-f.7 against A30_3 10 issues + repo truth (code, migrations, logs, rl9). All patches are small, deterministic cleanups that finish the f.7 specification without redesign:

#	A30_3 issue	Fix applied in PART 1 / Log	PART 1 §
1	Traffic-tiering called “Frozen” but lists OR options	Renamed table to “Candidate value (pending freeze at manifest — NOT frozen until reference data obtained)” and clarified final metric/source/period/threshold must be chosen once at freeze	§4.1
2	Six-region definition overlaps (NA vs SA Central America, AP vs OC Australia) + Russia country→region 1:1 conflict	Fixed regions: NA = north of 7°N, SA = S-only no Central America, AP = excluding AU/NZ/Pacific, OC = AU/NZ/Pacific only; added explicit <code>region_overrides</code> table with Russia 60°E longitude split	§4.2

#	A30_3 issue	Fix applied in PART 1 / Log	PART 1 §
3	T = scheduled_gate_out called frozen and unverified simultaneously	Changed §6.0 to “CANDIDATE pending Gate-0.5 verification” with fallback <code>scheduled_wheels_off</code> if unverifiable	§6.0
4	Retime contradiction (same id vs new id at 2h)	Harmonized §6.0 and §7.1: <code><2h → same flight_instance_id, 2h or date shift → new id with retime_parent_id</code>	§6.0 + §7.1
5	919 “worst case” not enforced (75 retry units only planned, not capped)	Added enforceable <code>FIDS_RETRY_UNIT_BUDGET=75</code> global counter; corrected math to <code>824 + 75 = 899 < 1,000</code> (was 919)	§5.4
6	<code>r_i</code> undefined in REGIONAL formula	Removed <code>r_i</code> ; formula is now <code>score_i = traffic_prior · m_i</code> capped at <code>traffic_prior · 1.5</code>	§8.2
7	Two incompatible next-action lists (Log §1 canary-before-anchor vs Plan §17 anchor-before-canary)	Made Log §1 authoritative matching Plan §17; Step 6 labeled “SMOKE canary (pre-gate, proves fix)” and added Step 10b official Gate-3 after Gate 2	Log §1 + Plan §17

#	A30_3 issue	Fix applied in PART 1 / Log	PART 1 §
8	“Phase 0 DONE / all implemented” too strong vs stub audit	Changed to “Legacy R1–R7/S1–S5 foundation complete; f.7/f.8 deltas DOCU- MENTED+STUB NOT LIVE- VERIFIED”	Log §3 + §15
9	A/B/C/D counts disagree (10 vs 15 C)	Made 77-row adjudication table the single source of truth; summaries generated from it	Log §19
10	Chain completeness / his- tory_ready_at / primary target fallback vague	Fixed chain denominator to distinguish scheduled vs observable successor, history_ready_at to operational full-lookback criterion, primary target fallback predeclared	§12.2.1 + §12.2.2 + §13.2
Stray	Trailing backtick after §6.0 candidate-T paragraph	Removed stray ‘	§6.0
Math	919 vs 899 headroom	Corrected to 899 with 101 headroom	§5.4

No new architecture, no new Gates, no Phase-6 start — purely a V3.9-f.7 consistency cleanup → **V3.9-f.8**. Plan header and Log §0.1 bumped to f.8; stray backtick removed; budget math unified.